

# The Design and Formal Analysis of the Dual Key Tunneling Protocol

John G. Underhill  
Quantum Resistant Cryptographic Solutions Corporation

June 2026

## Abstract

The Dual Key Tunneling Protocol is a post-quantum secure channel mechanism that combines asymmetric key exchange, digital signatures, pre-shared keys, a rolling transcript hash, and state-evolving key-update mechanisms to provide authenticated confidentiality and integrity for long-lived connections. The protocol uses a flexible construction that supports multiple post-quantum key encapsulation and signature schemes, including lattice-based families such as ML-KEM (formerly Kyber) and ML-DSA (formerly Dilithium) and optional code-based or hash-based alternatives such as Classic McEliece and SLH-DSA. Directional pre-shared keys may be configured to strengthen the tunnel establishment process and to provide hybrid symmetric protection that remains effective even if public key primitives are weakened in the future.

This paper provides a formal cryptanalysis of the implemented protocol profile. It begins with an engineering level specification of all handshake messages, key schedule components, transcript hashing rules, and transport phase operations, including the derivation of directional tunnel keys, initial nonces, and KDF-derived initial nonce construction and RCS internal counter progression. The description is written to enable faithful verification of all state transitions, message formats, and cryptographic inputs.

The paper then presents a formal model of DKTP that includes long-term keys, configurable pre-shared keys, ephemeral key pairs, ratchet states, and the complete transcript driven key schedule. A game-based adversarial model is defined with support for message manipulation, compromise of various key classes, quantum oracle queries to Keccak-based functions, and key indistinguishability tests. Security goals include authentication, handshake confidentiality, ciphertext integrity, replay resistance, forward secrecy, post-compromise security, and ratchet security.

For each goal we provide a conditional reduction to the underlying post-quantum assumptions, including IND-CCA security of the selected key encapsulation mechanism, existential unforgeability of the selected signature scheme, pseudo-randomness of the domain separated Keccak-based key derivation functions, and confidentiality and integrity of the RCS authenticated encryption system. The proofs include a transcript integrity lemma that incorporates pre-shared key inputs and formalizes the rolling transcript hash that binds all handshake messages into a single authenticated chain verified at tunnel establishment.

The formal analysis is complemented by an implementation conformance section that verifies that the state machine, message formats, KDF inputs, ratchet behavior, and failure handling in the reference implementation correspond to the formal model. A concrete security section provides security level estimates for typical parameter selections and quantifies the contribution of pre-shared keys. The results show that, under the stated assumptions and implementation constraints, DKTP provides a robust secure channel mechanism with conservative post-quantum design margins.

# 1 Introduction

The Dual Key Tunneling Protocol is a post-quantum secure channel protocol designed to establish long-lived, stateful, authenticated, and confidential tunnels between two peers. DKTP combines post-quantum public key exchange, post-quantum signatures, pre-shared symmetric keys, a rolling transcript hash binding mechanism, and a state-evolving tunnel key ratchet. The design emphasizes long-term robustness, flexibility in the choice of cryptographic primitives, resistance to quantum-capable adversaries under stated primitive assumptions, and precise control over session state transitions.

The protocol described in this paper follows the mechanism implemented in the DKTP reference code and documented in the DKTP technical specification. All handshake fields, key derivation inputs, associated data structures, replay protections, and ratcheting behaviors are drawn directly from those documents. The purpose of this formal analysis is to provide a rigorous, mathematically grounded foundation for the protocol as deployed, not for a simplified or idealized variant.

## 1.1 Background and Motivation

The transition from classical to post-quantum cryptography presents several challenges for secure channel protocols. Classical authenticated key exchange protocols often rely on mathematical assumptions that are no longer viable under quantum adversaries. Post quantum alternatives exist, but many have different performance characteristics, different security reductions, and different requirements for key management.

DKTP addresses these issues by providing a modular construction that supports multiple post-quantum key encapsulation mechanisms and digital signatures. The protocol does not assume a single fixed suite of primitives. Instead, it is compatible with a range of algorithms, including lattice-based systems such as ML-KEM (formerly Kyber) and ML-DSA (formerly Dilithium), code-based systems such as Classic McEliece, and hash-based systems such as SLH-DSA. Implementations select parameter sets at compile-time through configuration and do not alter the structure of the protocol.

A second motivation is resilience in the presence of long-term compromise. Many systems rely solely on public key material during the handshake. DKTP supplements this with directional pre-shared symmetric keys. These keys provide additional entropy, improve resistance to downgrade attacks, strengthen session establishment when long-term static keys are exposed, and contribute to recovery only when later handshakes or the optional asymmetric-ratchet path introduce fresh uncompromised KEM material and correctly persist the updated state.

Finally, DKTP is designed for systems that require stable stateful tunnels rather than single pass exchanges. The protocol integrates a tunnel level key derivation mechanism, per direction sequence numbers, authenticated timestamps, and key-update mechanisms that refresh symmetric and persisted PSK state at handshake boundaries and, when enabled, through an in-tunnel asymmetric ratchet. This supports long-lived channel operation with periodic renewal of cryptographic state.

## 1.2 Design Objectives for DKTP

DKTP was created according to several engineering and cryptographic objectives.

- **Post-quantum security.** The protocol must be instantiated with primitives intended to remain secure against quantum-capable adversaries. In the primary Q1 model, adversaries may perform quantum computation against public parameters but interact with protocol oracles classically. The KEM must satisfy post-quantum IND-CCA security and the signature system must provide post-quantum existential unforgeability under chosen message attack.

- **Support for algorithm agility.** The protocol must remain functional when instantiated with any supported post-quantum KEM or signature scheme. This includes configurable options for ML-KEM, ML-DSA, Classic McEliece, and SLH-DSA, selectable by the implementation without changing the packet structure.
- **Use of directional pre-shared keys.** The protocol maintains local and remote pre-shared key fields. These values may be equal at initial key generation and become directionally separated after the first successful session. The handshake incorporates them into the KDF, and successful establishment persists the updated PSK state.
- **Transcript hash-based session binding.** The handshake uses a 512-bit rolling transcript hash that is initialized from the message hash of the first connect-request and updated at each subsequent handshake stage by hashing in the authenticated keying material exchanged. The final transcript hash is encrypted and compared during the establish phase, ensuring that both parties agree on the complete exchange history and preventing tampering, reflection, or message reordering attacks.
- **Symmetric tunnel keys with directional separation.** The tunnel uses separate symmetric keys for sending and receiving. These keys are derived from the shared secrets produced by the KEM, combined with the directional pre-shared keys through domain separated cSHAKE-512 key derivation with per-channel identity labels.
- **Replay and ordering guarantees.** All encrypted transport packets include a fixed-format 21-byte header serialized as flag, message length, sequence number, and UTC timestamp. This header is authenticated as RCS associated data, providing replay protection and ordering guarantees.
- **Secure implementation and state handling.** The protocol must be implementable in constant-time. Ephemeral secrets and intermediate keying material must be erased at or before completion of the corresponding handshake or ratchet operation. Sequence numbers must be strictly ordered, and RCS nonces must remain unique under KDF-derived initialization and internal counter progression.

These objectives shape the design and analysis of the protocol. They also guide the security model developed in this paper.

### 1.3 Contributions of this Paper

This paper provides the complete specification-accurate, formal cryptanalysis of DKTP. Its contributions are:

- A detailed engineering description of the DKTP handshake and transport phases based strictly on the technical specification, including session transcripts, directional pre-shared keys, KEM operations, signature validation, authenticated encryption, and state transitions.
- A formal model that captures all relevant components of the protocol, including long-term asymmetric keys, directional pre-shared keys, configuration identifiers, ephemeral KEM keys, the rolling transcript hash, shared secrets, tunnel keys, and ratchet values.
- Formal definitions of all security goals relevant to secure channel protocols, including authentication, confidentiality, integrity, replay resistance, forward secrecy, post-compromise security, and ratchet security.

- Rigorous reduction based proofs showing how each security goal follows from the assumptions on the underlying primitives. These reductions include explicit treatment of pre-shared keys and the rolling transcript hash, and accurately reflect the behavior of the actual protocol.
- A cryptanalytic evaluation of the protocol that describes its attack surfaces, explains why each attack is mitigated, and identifies the security dependencies introduced by the KEM, the signature scheme, the AEAD mode, and the pre-shared key mechanism.
- A concrete security analysis that quantifies the strength of the protocol under different algorithm selections and different pre-shared key lengths.
- An implementation conformance section that verifies the alignment between the formal model and the DKTP reference implementation, including packet formats, KDF inputs, calls to cryptographic libraries, and erasure behavior.

## 1.4 Relationship to the Specification and Implementation

This formal analysis is developed from the DKTP technical specification and the reference C implementation. Every handshake step, message field, key derivation input, packet-header binding, and pre-shared key update described here corresponds to the protocol behavior defined for DKTP.

By combining a detailed engineering description, a precise mathematical model, and complete security reductions, this document ensures that the DKTP protocol can be analyzed, implemented, and evaluated with confidence. The alignment among specification, implementation, and formal analysis provides a foundation for long-term use of DKTP in environments that require high security against classical and quantum adversaries.

# 2 Engineering Description of DKTP

This section provides a complete and specification accurate engineering level description of the Dual Key Tunneling Protocol. The purpose is to give a clear and detailed account of every component of the protocol before the formal model is introduced. All information in this section is drawn directly from the DKTP technical specification and the associated reference implementation.

DKTP establishes a secure channel between an initiator and a responder. The structure consists of a six-message post-quantum handshake (three round trips) followed by a stateful transport phase. The six wire messages are connect-request, connect-response, exchange-request, exchange-response, establish-request, and establish-response. The initiator’s processing of the establish-response is a terminal verification step (establish-verify) that completes the handshake locally and emits no further packet. The handshake authenticates the peers, agrees on configuration, derives directional shared secrets, incorporates pre-shared keys, and produces tunnel keys and initial nonces. The transport phase uses authenticated encryption with strict sequence numbers, timestamps, and flags to provide confidentiality, integrity, and replay protection.

## 2.1 Protocol Roles and State Machines

DKTP operates between two peers that take the roles of initiator  $I$  and responder  $R$ . The initiator opens the transport connection and sends the connect request. The responder accepts the transport connection and processes the incoming connect request. Each endpoint maintains an exchange flag, a transmit sequence counter, and a receive sequence counter. The exchange flag identifies the current handshake stage or the established transport state.

The effective protocol states are Idle, Handshake, and Established. In the Idle state no DKTP session is active. In the Handshake state the endpoint processes the fixed six-message exchange: connect request, connect response, exchange request, exchange response, establish request, and establish response. The initiator’s establish-verify step is the local verification of the establish-response and does not transmit an additional packet. In the Established state both peers have verified the encrypted transcript confirmation and have initialized independent transmit and receive RCS instances.

The optional asymmetric ratchet is a sub-protocol executed inside the Established state. It does not create a separate connection state. Ratchet processing re-keys one cipher direction at a time while the session remains established.

## 2.2 Long Term Keys and Configuration Parameters

Each peer has three classes of long-lived cryptographic material.

**Long-term verification key pair.** Each peer holds a static post-quantum signature key pair  $(vk, sk)$  used to sign and verify handshake messages. The selected signature scheme is configuration dependent and may use ML-DSA or SLH-DSA where supported by the build.

**Peer-key pre-shared secret.** Each local and remote peer-key structure contains one 32-byte pre-shared secret field. In key-exchange state, the local peer-key value is loaded into *pssl* and the remote peer-key value is loaded into *pssr*. At initial key generation the reference generator creates one random pre-shared secret and copies it into both peer-key structures. Therefore  $pssl = pssr$  before the first successful session for a freshly generated pair. After successful session establishment, each value is advanced separately and the two PSK states diverge.

**Peer-key identity binding.** The local peer key contains a remote peer identifier. This value is expected to equal the key identifier in the corresponding remote peer key. A connection attempt is rejected when this peer-key binding does not hold. Multi-peer key lookup and selection are application-layer responsibilities; the DKTP library connection path operates on one supplied local and remote peer-key pair.

**Configuration parameters.** Configuration parameters identify the selected KEM, signature scheme, hash family, and symmetric cipher. The DKTP configuration field is 36 bytes and contains a null-terminated ASCII configuration string with zero padding as required. The configuration string is included in the initial signed transcript input, and mismatched configurations cause the exchange to fail.

## 2.3 Core Primitives and Algorithm Agility

DKTP is designed to be agnostic to the underlying post-quantum primitives. Implementations select algorithms at compile-time through the configuration file. DKTP supports:

- **Key Encapsulation Mechanisms:** ML-KEM parameter sets, Classic McEliece parameter sets.
- **Signature Schemes:** ML-DSA parameter sets, SLH-DSA parameter sets.
- **Authenticated Encryption:** RCS, a wide-block Rijndael based authenticated encryption construction that uses cSHAKE for key schedule and KMAC style authentication.

- **Hash Function and KDF:** Keccak-based functions with explicit domain separation, used to derive tunnel keys, nonces, and updated PSKs.

The protocol structure does not change when the cryptographic algorithms are changed.

## 2.4 Handshake Phases in Engineering Terms

From an engineering perspective, the DKTP handshake is realized as a fixed sequence of six protocol messages exchanged over three round trips, followed by a local establish-verify step at the initiator. These messages authenticate peer identities, perform a bidirectional KEM exchange, derive directional tunnel keys, and then explicitly confirm establishment before the tunnel is treated as operational.

$M_1$  : Connect-Request

$M_2$  : Connect-Response

$M_3$  : Exchange-Request

$M_4$  : Exchange-Response

$M_5$  : Establish-Request

$M_6$  : Establish-Response

The initiator's terminal verification of  $M_6$  is referred to as *Establish-Verify*; it changes only local state and emits no packet.

**Connect-Request (M1).** The initiator transmits its key identity and configuration string and authenticates the message by signing a hash of the serialized packet header, key identity, and configuration string. The initiator initializes the transcript hash as  $\text{sch} \leftarrow \text{H}(\text{H}(\text{sph} \parallel \text{kid} \parallel \text{cfg}))$ .

**Connect-Response (M2).** The responder validates the initiator signature and message hash, initializes its own copy of the transcript hash from the verified message hash, generates an ephemeral KEM keypair, signs a hash of the encapsulation key and outbound packet header, and returns the encapsulation key with its signature. Both peers update the transcript hash:  $\text{sch} \leftarrow \text{H}(\text{sch} \parallel \text{H}(\text{sph} \parallel \text{ek}))$ .

**Exchange-Request (M3).** The initiator validates the responder signature, performs KEM encapsulation to the responder encapsulation key, generates its own ephemeral KEM keypair, and sends the ciphertext, its encapsulation key, and a signature over the hash of all three components with the packet header. The transcript hash is updated with the verified and outgoing message hashes.

**Exchange-Response (M4).** The responder validates the initiator signature, decapsulates the initiator ciphertext to obtain the first shared secret, encapsulates to the initiator encapsulation key to obtain the second shared secret, derives directional tunnel keys and nonces via cSHAKE-512, and updates pre-shared secrets during key derivation. The transcript hash is updated with the signed ciphertext hash.

**Establish-Request (M5).** Once both directional cipher instances are initialized, the initiator encrypts the current transcript hash directly under the transmit channel cipher. The serialized establish-request header is supplied as AEAD associated data. After encryption, the initiator updates the transcript hash with the outbound establish-request ciphertext.

**Establish-Response (M6).** The responder decrypts the establish-request ciphertext and compares the result to its local transcript hash. If the values match, it updates the transcript with the received ciphertext, encrypts the updated transcript under its transmit channel cipher, and sends the establish response.

**Establish-Verify (initiator validates  $M_6$ ).** The initiator decrypts the establish-response ciphertext under its receive cipher, using the establish-response header as associated data, and compares the decrypted value to its own current transcript hash. If the values match, the tunnel is established and operational. Peering keys and kex state are cleared. This step transmits no packet.

The tunnel is not treated as established solely by completion of the KEM exchange. Operational activation occurs only after both the establish-response and establish-verify comparisons succeed.

## 2.5 Key Schedule and Directional Separation

The DKTP key schedule is sequential and directionally labelled. Channel labels are expressed from the initiator perspective. The label `DKTP:TXKEY:V1` denotes the initiator-to-responder channel. The label `DKTP:RXKEY:V1` denotes the responder-to-initiator channel. The responder therefore uses the TX label for its receive cipher and the RX label for its transmit cipher.

Let *secl* be the initiator-generated KEM shared secret and let *secr* be the responder-generated KEM shared secret. The initiator computes

$$prnd_1 \leftarrow \text{cSHAKE512}(X = secl, N = \text{DKTP:TXKEY:V1}, S = pssr),$$

$$tck_{I \rightarrow R} = prnd_1[0..63], \quad n_{I \rightarrow R} = prnd_1[64..95],$$

then updates

$$pssl' \leftarrow \text{cSHAKE512}(X = pssl, N = \epsilon, S = prnd_1[0..31]).$$

It then computes

$$prnd_2 \leftarrow \text{cSHAKE512}(X = secr, N = \text{DKTP:RXKEY:V1}, S = pssl'),$$

$$tck_{R \rightarrow I} = prnd_2[0..63], \quad n_{R \rightarrow I} = prnd_2[64..95],$$

and updates

$$pssr' \leftarrow \text{cSHAKE512}(X = pssr, N = \epsilon, S = prnd_2[0..31]).$$

Each channel KDF output contributes a 64-byte RCS key and a 32-byte initial RCS nonce. The DKTP packet sequence number is not used as a nonce. It is authenticated in the packet header and enforces strict ordering.

The tunnel is activated only after encrypted transcript confirmation. The initiator encrypts the current transcript hash using the establish-request header as associated data. The responder decrypts and compares it to its local transcript hash, updates the transcript with the establish-request ciphertext, and returns the updated transcript under its transmit cipher. The initiator accepts the session only when the decrypted response equals its own updated transcript hash.

## 2.6 Transport Phase, Ratcheting, and Tunnel Operation

After establishment, DKTP transports application data through independent RCS transmit and receive instances. Each encrypted packet carries a 21-byte authenticated header serialized as

$$\text{sph} = \text{flag}[1] \parallel \text{msglen}[4] \parallel \text{sequence}[8] \parallel \text{utctime}[8].$$

All multi-byte integer fields are encoded in little-endian order. The serialized header is supplied to RCS as associated data for transport packets and establish-phase confirmation packets.

RCS nonce progression is internal to the cipher state. DKTP supplies a 32-byte initial nonce derived by cSHAKE-512. RCS then advances its internal counter as data is processed. Sequence numbers are authenticated header fields and are not encoded as RCS nonces.

When the optional asymmetric ratchet is enabled, a peer may invoke a two-message in-tunnel sub-protocol while the connection remains established. The initiating peer sends an encrypted and signed ratchet request containing fresh KEM material. The responder decrypts and verifies the request, decapsulates the ratchet shared secret, re-keys the corresponding receive direction, and sends an encrypted and signed response carrying a second KEM contribution. The initiator decapsulates the response and re-keys its receive direction. The ratchet therefore updates one cipher direction at a time and does not repeat the six-message handshake.

## 2.7 Replay Protection and Ordering Requirements

DKTP uses two checks to prevent replay and reordering. First, every inbound packet must carry a UTC timestamp within the configured validity threshold. The default threshold is 60 seconds. Second, the receiver accepts only a packet whose sequence number equals the current receive sequence plus one. Stale, repeated, skipped, and out-of-order packets are rejected.

The sequence number and timestamp are authenticated because they are part of the serialized packet header used either in signed handshake hashes or as RCS associated data. A network adversary cannot modify these fields without invalidating the signature or AEAD authentication tag.

## 2.8 Failure Handling, Zeroization, and Implementation Notes

A DKTP implementation rejects a handshake or transport packet when any authenticated input is malformed, out of sequence, outside the timestamp window, inconsistent with the expected message size, signed under the wrong key, or rejected by RCS authentication. Failure clears the active exchange state and tears down the connection.

Temporary shared secrets, KDF buffers, and Keccak state are erased after use. The responder erases temporary shared-secret material during exchange-response processing. The initiator retains its first KEM shared secret and ephemeral decapsulation key in key-exchange state until the establish phase completes and the key-exchange reset clears that state. The forward-secrecy claim is therefore scoped to completed sessions and to erasure at or before handshake completion.

The implementation defines a sequence terminator constant, but the active receive path enforces strict increment-by-one sequencing rather than a separate explicit wrap guard. Practical deployments may impose an independent session lifetime or packet-count limit.

## 3 Formal Preliminaries

This section introduces the notation, cryptographic primitives, and adversarial capabilities that form the basis of the security model for DKTP. The formalism presented here aligns with the



engineering description of the protocol and matches the state variables and operations found in the DKTP specification and reference implementation. All assumptions are stated in terms of standard post-quantum definitions and oracle based adversarial interaction.

### 3.1 Notation and Conventions

We use uppercase letters for keys and shared secrets, lowercase letters for messages, and bold symbols for algorithms. Byte strings are written as elements of  $\{0, 1\}^*$ . Concatenation is written as  $A \parallel B$ . For a hash or KDF with domain separation, we write

$$\text{Hash}_{\text{label}}(X), \text{KDF}_{\text{label}}(X),$$

to indicate that the internal function is the same but that a label string is pre-pended within the domain separation space.

For a peer  $P$ , we denote its long-term signature key pair by  $(vk_P, sk_P)$  and its directional pre-shared keys by

$$\text{psk}_{P \rightarrow Q}, \text{psk}_{Q \rightarrow P}.$$

Each session has a configuration value `config` and a rolling transcript hash `sch` initialized from the message hash of the connect-request and updated at each subsequent handshake stage. The transcript hash is treated as a public but authenticated value that commits to the complete exchange history.

Random sampling from a distribution is written as  $x \leftarrow \mathcal{D}$ .

### 3.2 Post Quantum Primitives (General KEM and Signature Families)

DKTP supports a range of post-quantum key encapsulation and signature schemes. In the formal model, these are treated generically as families of algorithms with standard security properties. The specific implementation may select ML-KEM, Classic McEliece, or another IND-CCA secure KEM, and may select ML-DSA, SLH-DSA, or another EUF CMA secure signature scheme.

**Key encapsulation mechanism.** A post-quantum KEM consists of the algorithms

$$\begin{aligned} (pk, sk) &\leftarrow \text{KEM.KeyGen}(), \\ (c, K) &\leftarrow \text{KEM.Encap}(pk), \\ K &\leftarrow \text{KEM.Decap}(c, sk). \end{aligned}$$

The security requirement is IND-CCA security against classical adversaries and against quantum-capable adversaries in the selected post-quantum model. DKTP uses two encapsulations to derive directional shared secrets:

$$\text{secl}, \text{secl}.$$

**Signature scheme.** A post-quantum digital signature scheme consists of the algorithms

$$\begin{aligned} (vk, sk) &\leftarrow \text{SIG.KeyGen}(), \\ \sigma &\leftarrow \text{SIG.Sign}(sk, m), \\ \text{SIG.Verify}(vk, m, \sigma) &\in \{0, 1\}. \end{aligned}$$

The security requirement is existential unforgeability under chosen message attack. The Verification message in DKTP relies on this property.

### 3.3 Hash and KDF with Domain Separation

DKTP uses SHA3-512 for fixed-length hashing and cSHAKE-512 for domain-separated key expansion. The handshake KDF uses the KEM shared secret as input string  $X$ , the channel identity as function-name string  $N$ , and the current pre-shared key as customization string  $S$ :

$$\text{KDF}_{TX}(s, p) = \text{cSHAKE512}(X = s, N = \text{DKTP:TXKEY:V1}, S = p),$$

$$\text{KDF}_{RX}(s, p) = \text{cSHAKE512}(X = s, N = \text{DKTP:RXKEY:V1}, S = p).$$

The output consumed by DKTP is 96 bytes: a 64-byte RCS key followed by a 32-byte initial RCS nonce.

The optional asymmetric ratchet uses a separate domain string,  $\text{QRCS:PDEF:DKTP1A}$ . For ratchet shared secret  $ssec$  and current pre-shared secret  $pss$ ,

$$\begin{aligned} prnd &\leftarrow \text{cSHAKE512}(X = ssec, N = \text{QRCS:PDEF:DKTP1A}, S = pss), \\ pss' &\leftarrow \text{cSHAKE512}(X = prnd[0..31], N = \epsilon, S = pss). \end{aligned} \quad \text{where}$$

$prnd[0..63]$  is the RCS key and  $prnd[64..95]$  is the initial nonce. The ratchet PSK update is  $pss' \leftarrow \text{cSHAKE512}(X = prnd[0..31], N = \epsilon, S = pss)$ . This differs from the handshake update, where the old PSK is the input string and the first 32 bytes of tunnel material are the customization string.

### 3.4 Pre-Shared Keys in the Formal Model

The formal model represents local and remote peer-key secrets as  $pssl$  and  $pssr$ . They are state variables. For a freshly generated peer-key pair they may originate from the same random 32-byte value. After a successful session they are updated separately and act as directional PSK state.

**Use in key derivation.** The PSK value is used as the cSHAKE customization string in the channel KDF. For the initiator-to-responder direction, the initiator uses  $pssr$  and the responder uses  $pssl$  with the TX label. For the responder-to-initiator direction, the initiator uses updated  $pssl$  and the responder uses updated  $pssr$  with the RX label.

**Handshake update rule.** The handshake PSK update is

$$pss' \leftarrow \text{cSHAKE512}(X = pss, N = \epsilon, S = tck[0..31]),$$

where  $tck[0..31]$  is the first 32 bytes of the corresponding KDF output block.

**Compromise considerations.** Security claims involving PSK secrecy are conditioned on secure storage and correct persistence of the peer-key state. Disclosure before the first successful session reveals the single initial PSK value copied into both peer keys. Disclosure after session establishment reveals the current persisted state for that pairing.

### 3.5 Assumptions and Adversarial Interaction

The adversary  $\mathcal{A}$  controls all network communication. It may delay, reorder, inject, or modify messages. The adversary interacts with instances of the protocol through oracles that allow the creation of sessions, message delivery, and key exposure.

The main assumptions are:

- KEM security: IND-CCA under quantum adversaries.
- Signature security: EUF CMA under classical and quantum adversaries.
- KDF security: pseudo-randomness of the domain separated Keccak-based construction.
- AEAD security: confidentiality and integrity of RCS.
- Pre-shared keys remain secret and uncompromised.

Implementation assumptions, such as secure erasure, constant-time KEM decapsulation, and CSPRNG quality, are stated separately in the implementation conformance section.

### 3.6 Oracle Definitions

The adversary interacts with the protocol through the following oracles. Each oracle operates on a session  $(P, \text{sid})$  belonging to party  $P$ .

- **Send** $(P, \text{sid}, m)$ . Delivers message  $m$  to the local protocol instance. Used to initiate new sessions and to inject handshake and transport packets.
- **RevealEPH** $(P, \text{sid})$ . Reveals the ephemeral KEM private key used in the handshake for that session. This oracle is restricted by freshness conditions in the security games.
- **RevealTunnelKey** $(P, \text{sid})$ . Reveals the directional tunnel key for a completed session. Forbidden for sessions that will be tested in confidentiality games.
- **Corrupt** $(P)$ . Reveals the long-term signature secret key of party  $P$ . Permitted in some games and forbidden in others depending on the security goal.
- **Test** $(P, \text{sid})$ . Used in confidentiality, forward secrecy, post-compromise security, and ratchet security games. Returns either the real key or a random key depending on a hidden bit chosen by the challenger.

These oracle definitions form the basis for all security games in later chapters. They accurately reflect the operations that the implementation performs and the ways in which an adversary might attempt to gain information or influence session state.

## 4 Formal Protocol Specification

This section presents a precise, specification accurate formal description of the DKTP protocol. Every field, operation, and state component is taken directly from the technical specification and the reference implementation. The handshake, transcript binding, key schedule, tunnel derivation, and ratchet evolution are given in a mathematically explicit form suitable for later security analysis.

### 4.1 Formal Handshake Messages

This subsection defines the DKTP handshake message flow in a form suitable for security analysis, matching the protocol specified in the DKTP Specification. The handshake consists of six messages exchanged between an initiator  $I$  and a responder  $R$  over three round trips, followed by the initiator's local establish-verify step.

**Long-term keys and header binding.** Each peer possesses a long-term signature keypair  $(sk_X^{sig}, vk_X^{sig})$  for  $X \in \{I, R\}$ . Handshake authentication binds the serialized packet header **Hdr** (including sequence number and timestamp) to the message payload by signing a hash of  $(\text{payload} \parallel \text{Hdr})$ .

**Message M1: Connect-Request** ( $I \rightarrow R$ ). The initiator sends its key identity and configuration string, authenticated under its signature key. We model the payload as

$$M_1 = (\text{kid}, \text{cfg}),$$

and the transmitted message includes a signature over the message hash bound to the packet header:

$$\sigma_1 = \text{Sig.Sign}(sk_I^{sig}, \text{Hash}(\text{sph} \parallel \text{kid} \parallel \text{cfg})).$$

The initiator initializes the transcript hash:  $\text{sch} \leftarrow H(H(\text{sph} \parallel \text{kid} \parallel \text{cfg}))$ .

**Message M2: Connect-Response** ( $R \rightarrow I$ ). The responder verifies  $\sigma_1$  and the message hash, initializes its own transcript hash  $\text{sch} \leftarrow H(\text{hm})$  from the verified message hash, and generates an ephemeral KEM keypair:

$$(\text{dk}_R, \text{ek}_R) \leftarrow \text{KEM.Gen.}$$

The payload is modeled as

$$M_2 = (\text{ek}_R),$$

and the responder signs the hash of the encapsulation key bound to the outbound packet header:

$$\sigma_2 = \text{Sig.Sign}(sk_R^{sig}, \text{Hash}(\text{sph} \parallel \text{ek}_R)).$$

Both peers update the transcript hash:  $\text{sch} \leftarrow H(\text{sch} \parallel H(\text{sph} \parallel \text{ek}_R))$ .

**Message M3: Exchange-Request** ( $I \rightarrow R$ ). The initiator verifies  $\sigma_2$  and the encapsulation key hash, then performs KEM encapsulation to  $\text{ek}_R$  obtaining the local shared secret  $\text{secl}$  and ciphertext  $c_{pta}$ :

$$(c_{pta}, \text{secl}) \leftarrow \text{KEM.Encap}(\text{ek}_R).$$

It also generates its own ephemeral KEM keypair  $(\text{dk}_I, \text{ek}_I) \leftarrow \text{KEM.Gen}()$  and stores  $\text{dk}_I$ . The payload is modeled as

$$M_3 = (c_{pta}, \text{ek}_I),$$

with signature over the hash of the ciphertext, encapsulation key, and exchange-request packet header:

$$\sigma_3 = \text{Sig.Sign}(sk_I^{sig}, \text{Hash}(\text{sph} \parallel c_{pta} \parallel \text{ek}_I)).$$

The transcript hash is updated:  $\text{sch} \leftarrow H(\text{sch} \parallel H(\text{sph} \parallel c_{pta} \parallel \text{ek}_I))$ .

**Message M4: Exchange-Response** ( $R \rightarrow I$ ). The responder verifies  $\sigma_3$  and decapsulates  $c_{pta}$  to obtain the remote shared secret  $\text{secl}$ . The responder encapsulates to  $\text{ek}_I$  to obtain the local shared secret  $\text{secl}$  and ciphertext  $c_{ptb}$ :

$$(c_{ptb}, \text{secl}) \leftarrow \text{KEM.Encap}(\text{ek}_I).$$

The responder derives directional tunnel keys and nonces via cSHAKE-512 as in Section 4.5, updates pre-shared secrets inline, initializes cipher instances, and returns  $c_{ptb}$  authenticated under its signature:

$$M_4 = (c_{ptb}), \quad \sigma_4 = \text{Sig.Sign}(\text{sk}_R^{\text{sig}}, \text{Hash}(\text{sph} \parallel c_{ptb})).$$

The transcript hash is updated:  $\text{sch} \leftarrow H(\text{sch} \parallel H(\text{sph} \parallel c_{ptb}))$ . The initiator verifies  $\sigma_4$  and decapsulates  $c_{ptb}$  to obtain its copy of  $\text{secr}$ , then derives the same directional tunnel keys and initializes cipher instances.

**Message M5: Establish-Request ( $I \rightarrow R$ ).** The initiator validates  $M_4$ , decapsulates the second shared secret, derives the RCS contexts, and encrypts the current transcript hash directly under its transmit cipher. The establish-request header is used as associated data:

$$\text{cpt}_5 \leftarrow \text{RCS.Enc}_{tck_{I \rightarrow R}}(\text{sch}; AD = \text{sph}_5).$$

After encryption, the initiator updates its transcript:

$$\text{sch} \leftarrow H(\text{sch} \parallel \text{cpt}_5).$$

The transmitted message is  $M_5 = \text{cpt}_5$ .

**Message M6: Establish-Response ( $R \rightarrow I$ ).** The responder decrypts  $M_5$  using the establish-request header as associated data and compares the decrypted value to its local transcript hash. On success it updates the transcript with the received ciphertext and encrypts the updated transcript using the establish-response header as associated data:

$$\text{sch} \leftarrow H(\text{sch} \parallel \text{cpt}_5),$$

$$\text{cpt}_6 \leftarrow \text{RCS.Enc}_{tck_{R \rightarrow I}}(\text{sch}; AD = \text{sph}_6).$$

The transmitted message is  $M_6 = \text{cpt}_6$ . No further PSK update occurs during the establish phase.

**Establish-Verify ( $I$  validates  $M_6$ ).** The initiator decrypts  $M_6$  using the establish-response header as associated data and compares the decrypted value to its current transcript hash:

$$\text{RCS.Dec}_{tck_{R \rightarrow I}}(\text{cpt}_6; AD = \text{sph}_6) \stackrel{?}{=} \text{sch}.$$

Here  $\text{sch}$  is the initiator's transcript value after the establish-request update  $\text{sch} \leftarrow H(\text{sch} \parallel \text{cpt}_5)$ , which equals the responder's value after the symmetric update applied in  $M_6$ . If the comparison succeeds, both peers hold the same authenticated transcript and the session enters the established state. This step emits no packet.

## 4.2 Rolling Transcript Hash

DKTP binds the handshake transcript to the establishment confirmation using a 512-bit rolling transcript hash  $\text{sch}$ . The value  $\text{sch}$  is initialized from the message hash of the connect-request and updated at each subsequent handshake stage by absorbing the authenticated keying material exchanged. At the establish phase,  $\text{sch}$  is encrypted and compared between both peers to confirm that both sides observed an identical exchange history.

**Transcript hash initialization.** The initiator initializes  $\text{sch}$  by hashing the message hash of the connect-request:

$$\text{hm} \leftarrow H(\text{sph} \parallel \text{kid} \parallel \text{cfg}), \quad \text{sch} \leftarrow H(\text{hm}).$$

The responder initializes its own copy of  $\text{sch}$  from the same verified message hash after validating the connect-request signature. No verification keys are included in this initialization.

**Transcript hash updates.** After each handshake message,  $\text{sch}$  is updated by hashing in the authenticated material from that stage:

$$\text{sch} \leftarrow H(\text{sch} \parallel h_{\text{msg}}),$$

where  $h_{\text{msg}}$  is the hash of the authenticated payload of the current message (e.g.,  $H(\text{sph} \parallel \text{ek})$  for the connect-response encapsulation key). This accumulation continues through messages  $M_2$  through  $M_4$ .

**Establishment binding.** During establishment the transcript hash itself is used as the confirmation value; DKTP does not compute a separate keyed confirmation digest. The initiator encrypts the current transcript hash  $\text{sch}$  directly under the transmit channel cipher, supplying the serialized establish-request header  $\text{sph}_5$  as RCS associated data:

$$\text{cpt}_5 \leftarrow \text{RCS.Enc}_{tck_{I \rightarrow R}}(\text{sch}; AD = \text{sph}_5).$$

The per-packet sequence number and timestamp are bound to the confirmation through the AEAD tag over  $\text{sph}_5$ , not through an additional hash. After producing  $\text{cpt}_5$ , the initiator advances the transcript by absorbing the ciphertext,

$$\text{sch} \leftarrow H(\text{sch} \parallel \text{cpt}_5),$$

and the responder performs the identical update after decrypting and verifying  $\text{cpt}_5$ , so both endpoints hold the same transcript value before the establish-response is sent. The responder then encrypts this updated value under  $tck_{R \rightarrow I}$  with  $\text{sph}_6$  as associated data, and the initiator accepts only if the decrypted establish-response equals its own updated transcript.

Because the AEAD associated data binds the per-packet sequence number and timestamp, and because  $\text{sch}$  accumulates all authenticated handshake material from  $M_1$  through  $M_4$  together with the establish-request ciphertext, the establishment confirmation prevents replay, transcript splicing, message reordering, and cross-session attacks. The fixed ratchet domain string  $\text{QRCS:PDEF:DKTP1A}$  is used only by the asymmetric-ratchet KDF (Section 4.5 and the ratchet analysis) and is *not* mixed into the establishment confirmation.

The transcript hash is not a secret. Its purpose is to provide transcript binding and explicit confirmation that both peers derived identical tunnel state from an identical exchange history before the tunnel is considered established.

### 4.3 Formal Key Schedule (Including PSK Mixing)

Let  $\text{secl}$  be the initiator-generated KEM shared secret and  $\text{secr}$  the responder-generated KEM shared secret. The initiator computes

$$\text{prnd}_1 \leftarrow \text{cSHAKE512}(X = \text{secl}, N = \text{DKTP:TXKEY:V1}, S = \text{psr}),$$

$$tck_{I \rightarrow R} = \text{prnd}_1[0..63], \quad n_{I \rightarrow R} = \text{prnd}_1[64..95],$$

$$\begin{aligned}
pssl' &\leftarrow \text{cSHAKE512}(X = pssl, N = \epsilon, S = prnd_1[0..31]), \\
prnd_2 &\leftarrow \text{cSHAKE512}(X = secr, N = \text{DKTP:RXKEY:V1}, S = pssl'), \\
tck_{R \rightarrow I} &= prnd_2[0..63], \quad n_{R \rightarrow I} = prnd_2[64..95], \\
pssr' &\leftarrow \text{cSHAKE512}(X = pssr, N = \epsilon, S = prnd_2[0..31]).
\end{aligned}$$

The responder derives the same channel values using the same client-perspective labels, but installs  $tck_{I \rightarrow R}$  in its receive cipher and  $tck_{R \rightarrow I}$  in its transmit cipher. The second derivation depends on the updated  $pssl'$  value and is analyzed as a sequential PRF composition.

#### 4.4 Security Margin and Long-Horizon Threat Model

This subsection clarifies the threat model and design rationale underlying DKTP’s choice of cryptographic parameters. It does not introduce additional security assumptions beyond those already used in the formal analysis, but explains the intended security horizon and margin considerations.

**Long-horizon adversary model.** DKTP is designed for scenarios in which confidentiality and integrity of communications are expected to remain robust over extended time horizons, including periods in which cryptanalytic techniques, computational capabilities, or implementation attack surfaces may evolve in unpredictable ways. The design therefore adopts conservative parameter choices that exceed those commonly used in deployed secure channel protocols.

**Symmetric security margin.** DKTP instantiates its transport channel using RCS-512, a 512-bit keyed authenticated encryption scheme with large nonce space and configurable authentication tag length. Directional channel keys are derived independently for each communication direction, yielding two cryptographically independent channel instances per session.

The intent of this construction is not to claim information-theoretic security, but to provision a substantial security margin against: (i) generic brute-force search, (ii) quantum speedups such as Grover-style attacks, and (iii) unforeseen advances in cryptanalysis affecting symmetric primitives.

Consistent with the RCS security analysis, classical security margins scale with the symmetric key length and sponge capacity, while post-quantum security is conservatively estimated under generic quadratic speedup assumptions.

**Integrity margin and misuse resistance.** In addition to key length, DKTP emphasizes large authentication margins and strict nonce discipline. Authentication tags and MAC constructions are selected to significantly exceed conventional minimum sizes, reducing the probability of successful forgery even under high query volumes or partial state exposure.

**Relation to compromise protection.** Extended symmetric margins complement, but do not replace, other compromise-mitigation mechanisms in DKTP, including: ephemeral key exchange, explicit establishment confirmation, directional separation of channel state, evolving persistent secret material across sessions, and mandatory erasure of ephemeral secrets after establishment.

The overall security of DKTP therefore derives from a combination of conservative cryptographic margins and strict state lifecycle management, rather than reliance on any single primitive or assumption.

**Scope of claims.** All security claims in this document remain computational in nature and are conditioned on the assumed hardness of the underlying primitives and the correct implementation of the protocol. DKTP does not claim detection of passive eavesdropping or information-theoretic secrecy, but aims to approach the upper bound of achievable security guarantees within a purely software-based cryptographic framework.

#### 4.5 RCS Key and Nonce Derivation

Each channel consumes 96 bytes from the cSHAKE-512 output. Bytes 0 through 63 form the RCS-512 key. Bytes 64 through 95 form the 32-byte initial RCS nonce. RCS is initialized once per direction during the handshake.

The RCS nonce is not constructed from the DKTP packet sequence number. Nonce uniqueness relies on the KDF-derived initial nonce and RCS internal counter progression. The sequence number is an authenticated packet-header field used for strict ordering and replay rejection.

#### 4.6 Ratchet Chains and State Evolution

The optional asymmetric ratchet is a two-message sub-protocol carried inside the encrypted tunnel. Each KEM operation updates one cipher direction. For ratchet shared secret  $ssec$  and current PSK  $pss$ ,

$$\begin{aligned} prnd &\leftarrow \text{cSHAKE512}(X = ssec, N = \text{QRCS:PDEF:DKTP1A}, S = pss), \\ tck &= prnd[0..63], \quad n = prnd[64..95], \\ pss' &\leftarrow \text{cSHAKE512}(X = prnd[0..31], N = \epsilon, S = pss). \end{aligned}$$

The request updates the initiator transmit direction and responder receive direction. The response updates the responder transmit direction and initiator receive direction. The connection remains established throughout this sub-protocol.

#### 4.7 Transport Encryption and Associated Data Structure

Every encrypted DKTP packet authenticates its serialized packet header. The header is

$$\text{sph} = \text{flag}[1] \parallel \text{msglen}[4] \parallel \text{sequence}[8] \parallel \text{utctime}[8].$$

The integer fields are little-endian. RCS encryption is modeled as

$$\text{cpt} \leftarrow \text{RCS.Enc}_{tck}(m; AD = \text{sph}).$$

The receiver accepts only if RCS authentication succeeds, the expected flag and message size match, the timestamp is valid, and the sequence number equals the current receive sequence plus one.

#### 4.8 Handshake Pseudo-code Definitions

**Client: Connect Request.** This function constructs the initial Connect Request message from the client. It verifies that the long-term key material has not expired, assembles the key identity and configuration fields, computes the session transcript hash from the serialized header, key identity, and configuration string, and signs the header and payload using the client long-term signature key. The resulting packet contains no sensitive secret material and establishes the starting point for the authenticated handshake. The state variable  $\text{schash}$  is stored for later use in the establish confirmation step.



---

**Algorithm 1** CLIENTCONNECTREQUEST

---

**Require:** Client kex state  $kcs$ , connection state  $cns$

**Ensure:** Outgoing packet  $pkt$  of type `connect_request` or error

```
1:  $t \leftarrow \text{UTC\_now}()$ 
2: if  $t > kcs.\text{expiration}$  then
3:    $cns.\text{exflag} \leftarrow \text{none}$ 
4:   return key_expired
5: end if
6:  $pkt.\text{pmessage}[0..\text{DKTP\_KEYID\_SIZE}] \leftarrow kcs.\text{keyid}$ 
7:  $pkt.\text{pmessage}[\text{DKTP\_KEYID\_SIZE}..] \leftarrow \text{DKTP\_CONFIG\_STRING}$ 
8:  $mtype \leftarrow \text{connect\_request}$ 
9:  $mten \leftarrow \text{KEX\_CONNECT\_REQUEST\_MESSAGE\_SIZE}$ 
10:  $\text{HeaderCreate}(pkt, mtype, cns.\text{txseq}, mten)$ 
11:  $shdr \leftarrow \text{SerializeHeader}(pkt)$ 
12:  $phash \leftarrow \text{SHA3\_512}(shdr \parallel pkt.\text{pmessage}[0..\text{DKTP\_KEYID\_SIZE} + \text{DKTP\_CONFIG\_SIZE}])$ 
13:  $sig \leftarrow \text{Sign}(phash, kcs.\text{sigkey})$ 
14: Write  $sig$  into  $pkt.\text{pmessage}$  immediately after keyid and config
15:  $hm \leftarrow \text{SHA3\_512}(shdr \parallel kcs.\text{keyid} \parallel \text{DKTP\_CONFIG\_STRING})$ 
16:  $kcs.\text{shash} \leftarrow \text{SHA3\_512}(hm)$ 
17:  $cns.\text{exflag} \leftarrow \text{connect\_request}$ 
18: return none
```

---

**Server: Connect Response.** This function processes the client Connect Request and verifies that the configuration and signature fields are valid. It reconstructs the packet hash from the received header and payload and checks it against the client signature. It then initializes the session transcript hash from the verified message hash, generates a fresh ephemeral KEM key pair, and updates the transcript hash with the encapsulation key hash. The responder signs the encapsulation key hash bound to the outbound packet header and returns the encapsulation key with its signature. The server stores its copy of the session transcript hash for later establish confirmation.

---

**Algorithm 2** SERVERCONNECTRESPONSE

---

**Require:** Server kex state  $kss$ , connection state  $cns$ , incoming packet  $pkt_{in}$

**Ensure:** Outgoing packet  $pkt_{out}$  of type `connect_response` or error

```
1:  $t \leftarrow \text{UTC\_now}()$ 
2: if  $t > kss.\text{expiration}$  then
3:    $cns.\text{exflag} \leftarrow \text{none}$ 
4:   return key_expired
5: end if
6:  $\text{cfg} \leftarrow \text{config bytes from } pkt_{in}.\text{pmessage}$ 
7: if  $\text{cfg} \neq \text{DKTP\_CONFIG\_STRING}$  then
8:    $cns.\text{exflag} \leftarrow \text{none}$ 
9:   return unknown_protocol
10: end if
11:  $\text{sig} \parallel \text{hm} \leftarrow \text{tail of } pkt_{in}.\text{pmessage}$ 
12: if  $\neg \text{Verify}(\text{sig}, \text{hm}, kss.\text{rverkey})$  then
13:    $cns.\text{exflag} \leftarrow \text{none}$ 
14:   return authentication_failure
15: end if
16:  $\text{shdr} \leftarrow \text{SerializeHeader}(pkt_{in})$ 
17:  $\text{hmc} \leftarrow \text{SHA3\_512}(\text{shdr} \parallel \text{kid} \parallel \text{cfg})$ 
18: if  $\text{VerifyEqual}(\text{hm}, \text{hmc}) \neq 0$  then
19:    $cns.\text{exflag} \leftarrow \text{none}$ 
20:   return verify_failure
21: end if
22:  $\text{hm} \leftarrow \text{SHA3\_512}(\text{shdr} \parallel \text{kid} \parallel \text{cfg})$  // Same message hash verified from M1
23:  $kss.\text{schash} \leftarrow \text{SHA3\_512}(\text{hm})$ 
24:  $(kss.\text{enckey}, kss.\text{deckey}) \leftarrow \text{KEM.KeyGen}()$ 
25:  $\text{mtype} \leftarrow \text{connect\_response}$ 
26:  $\text{mlen} \leftarrow \text{KEX\_CONNECT\_RESPONSE\_MESSAGE\_SIZE}$ 
27:  $\text{HeaderCreate}(pkt_{out}, \text{mtype}, \text{cns.txseq}, \text{mlen})$ 
28:  $\text{shdr}' \leftarrow \text{SerializeHeader}(pkt_{out})$ 
29:  $\text{phash} \leftarrow \text{SHA3\_512}(\text{shdr}' \parallel kss.\text{enckey})$ 
30: // Update transcript hash with encapsulation key
31:  $kss.\text{schash} \leftarrow \text{SHA3\_512}(kss.\text{schash} \parallel \text{phash})$ 
32:  $\text{sig}' \leftarrow \text{Sign}(\text{phash}, kss.\text{sigkey})$ 
33: Write  $\text{sig}'$  then  $kss.\text{enckey}$  into  $pkt_{out}.\text{pmessage}$ 
34:  $cns.\text{exflag} \leftarrow \text{connect\_response}$ 
35: return none
```

---

**Client: Exchange Request.** This function processes the server Connect Response and authenticates the responder through its long-term verification key. It reconstructs the expected header hash, verifies the signature, and then performs encapsulation against the server ephemeral KEM public key to derive the client to server shared secret. The client generates its own ephemeral KEM key pair and sends both the ciphertext and its ephemeral public key to the server. This message includes a signature that authenticates the client contribution to the handshake transcript. The function stores the first KEM shared secret for use in the key derivation and ratchet steps.

---

**Algorithm 3** CLIENTEXCHANGEREQUEST

---

**Require:** Client kex state  $kcs$ , connection state  $cns$ , server packet  $pkt_{in}$  (connect response)

**Ensure:** Outgoing packet  $pkt_{out}$  (exchange request) or error

```
1: Parse  $pkt_{in}.pmessage$  as
                                 $sig \parallel hm \parallel pubk$ 

   where  $pubk$  is server KEM public key
2: if  $\neg \text{Verify}(sig, hm, kcs.rverkey)$  then
3:    $cns.exflag \leftarrow \text{none}$ 
4:   return authentication_failure
5: end if
6:  $shdr \leftarrow \text{SerializeHeader}(pkt_{in})$ 
7:  $hmc \leftarrow \text{SHA3\_512}(shdr \parallel pubk)$ 
8: if  $\text{VerifyEqual}(hmc, hm) \neq 0$  then
9:    $cns.exflag \leftarrow \text{none}$ 
10:  return verify_failure
11: end if
12:  // Update transcript hash with verified M2 encapsulation key
13:  $kcs.schash \leftarrow \text{SHA3\_512}(kcs.schash \parallel hmc)$ 
14:  $(kcs.secl, cpta) \leftarrow \text{KEM.Encapsulate}(pubk)$ 
15:  $(kcs.enckey, kcs.deckey) \leftarrow \text{KEM.KeyGen}()$ 
16: Write  $cpta$  then  $kcs.enckey$  into  $pkt_{out}.pmessage$ 
17:  $mtype \leftarrow \text{exchange\_request}$ 
18:  $melen \leftarrow \text{KEX\_EXCHANGE\_REQUEST\_MESSAGE\_SIZE}$ 
19:  $\text{HeaderCreate}(pkt_{out}, mtype, cns.txseq, melen)$ 
20:  $shdr' \leftarrow \text{SerializeHeader}(pkt_{out})$ 
21:  $phash \leftarrow \text{SHA3\_512}(shdr' \parallel cpta \parallel kcs.enckey)$ 
22:  $sig' \leftarrow \text{Sign}(phash, kcs.sigkey)$ 
23: Append  $sig'$  to  $pkt_{out}.pmessage$ 
24:  // Update transcript hash with outgoing M3 message hash
25:  $kcs.schash \leftarrow \text{SHA3\_512}(kcs.schash \parallel phash)$ 
26:  $cns.exflag \leftarrow \text{exchange\_request}$ 
27: return none
```

---

**Server: Exchange Response And Ratchet.** This function processes the client Exchange Request and completes the asymmetric key agreement. It verifies the client signature and computes the shared secret  $secl$  by decapsulating the ciphertext received from the client. It then encapsulates to the client ephemeral key to compute the second shared secret  $secl$ . With both directional shared secrets available, the function computes the directional tunnel keys and initial nonces through cSHAKE based expansion and updates both directional pre-shared keys using the DKTP PSK update rule. It initializes the RCS contexts for transmitting and receiving and prepares the Exchange Response containing the second ciphertext and the server signature. This function performs the full server side ratchet and symmetric state initialization.

---

**Algorithm 4** SERVEREXCHANGEREESPONSEANDRATCHET

---

**Require:** Server kex state  $kss$ , connection state  $cns$ , client packet  $pkt_{in}$  (exchange request)

**Ensure:** Outgoing packet  $pkt_{out}$  (exchange response) or error

```
1: Parse  $pkt_{in}.pmessage$  as
                                 $cpta \parallel pubk_C \parallel sig$ 
   where  $cpta$  is client ciphertext,  $pubk_C$  is client KEM public key
2: if  $\neg \text{Verify}(sig, khash, kss.rverkey)$  then
3:    $cns.exflag \leftarrow \text{none}$ 
4:   return authentication_failure
5: end if
6:  $shdr \leftarrow \text{SerializeHeader}(pkt_{in})$ 
7:  $phash \leftarrow \text{SHA3\_512}(shdr \parallel cpta \parallel pubk_C)$ 
8: if  $\text{VerifyEqual}(phash, khash) \neq 0$  then
9:    $cns.exflag \leftarrow \text{none}$ 
10:  return hash_invalid
11: end if
12:  // Update transcript hash with verified M3 message hash
13:  $kss.schash \leftarrow \text{SHA3\_512}(kss.schash \parallel phash)$ 
14:  $secl \leftarrow \text{KEM.Decapsulate}(cpta, kss.deckey)$ 
15: if decapsulation fails then
16:    $cns.exflag \leftarrow \text{none}$ 
17:   return decapsulation_failure
18: end if
19:  $(secl, cptb) \leftarrow \text{KEM.Encapsulate}(pubk_C)$ 
20: Write  $cptb$  into  $pkt_{out}.pmessage$ 
21:  $mttype \leftarrow \text{exchange\_response}$ 
22:  $mlen \leftarrow \text{KEX\_EXCHANGE\_RESPONSE\_MESSAGE\_SIZE}$ 
23:  $\text{HeaderCreate}(pkt_{out}, mttype, cns.txseq, mlen)$ 
24:  $shdr' \leftarrow \text{SerializeHeader}(pkt_{out})$ 
25:  $phash' \leftarrow \text{SHA3\_512}(shdr' \parallel cptb)$ 
26:  $sig' \leftarrow \text{Sign}(phash', kss.sigkey)$ 
27: Append  $sig'$  to  $pkt_{out}.pmessage$ 
28:  // Update transcript hash with outgoing M4 message hash
29:  $kss.schash \leftarrow \text{SHA3\_512}(kss.schash \parallel phash')$ 
30:  // Derive symmetric keys and perform PSK ratchet, server view
31:  $prnd \leftarrow \text{cSHAKE512\_XOF}(\text{input} = secl, \text{custom} = kss.pssl, 2 \text{ blocks})$ 
32:  $tck_{C \rightarrow S}.key \leftarrow prnd[0..63]$ 
33:  $tck_{C \rightarrow S}.nonce \leftarrow prnd[64..95]$ 
34: Initialize  $cns.rxcp$  as RCS with  $tck_{C \rightarrow S}$  in receive mode
35:  $kss.pssr \leftarrow \text{cSHAKE512}(kss.pssr, tck_{C \rightarrow S}.key[0..31])$ 
36:  $prnd \leftarrow \text{cSHAKE512\_XOF}(\text{input} = secl, \text{custom} = kss.pssr, 2 \text{ blocks})$ 
37:  $tck_{S \rightarrow C}.key \leftarrow prnd[0..63]$ 
38:  $tck_{S \rightarrow C}.nonce \leftarrow prnd[64..95]$ 
39: Initialize  $cns.txcp$  as RCS with  $tck_{S \rightarrow C}$  in transmit mode
40:  $kss.pssl \leftarrow \text{cSHAKE512}(kss.pssl, tck_{S \rightarrow C}.key[0..31])$ 
41:  $cns.exflag \leftarrow \text{exchange\_response}$ 
42: return none
```

---

**Client: Establish Request And Ratchet.** This function processes the server Exchange Response and completes the handshake from the client perspective. It verifies the server signature, decapsulates the ciphertext from the server to derive  $secl$ , and then computes the directional tunnel keys and initial nonces through the same cSHAKE based expansion used on the server. It initializes

the RCS contexts for transmitting and receiving, updates both directional pre-shared keys according to the PSK update rule, and constructs the Establish Request message. This message carries the current session transcript hash encrypted under the transmit cipher, with the establish-request header authenticated as associated data; after encryption the client advances its transcript by absorbing the ciphertext. This function performs the full client side ratchet and symmetric state initialization.

---

**Algorithm 5** CLIENTESTABLISHREQUESTANDRATCHET

---

**Require:** Client kex state  $kcs$ , connection state  $cns$ , server packet  $pkt_{in}$  (exchange response)

**Ensure:** Outgoing packet  $pkt_{out}$  (establish request) or error

```
1: Parse  $pkt_{in}.pmessage$  as
                                 $cptb \parallel sig$ 

2: if  $\neg \text{Verify}(sig, hm, kcs.rverkey)$  then
3:    $cns.exflag \leftarrow \text{none}$ 
4:   return authentication_failure
5: end if
6:  $shdr \leftarrow \text{SerializeHeader}(pkt_{in})$ 
7:  $hmc \leftarrow \text{SHA3\_512}(shdr \parallel cptb)$ 
8: if  $\text{VerifyEqual}(hmc, hm) \neq 0$  then
9:    $cns.exflag \leftarrow \text{none}$ 
10:  return verify_failure
11: end if
12:  // Update transcript hash with verified M4 message hash
13:  $kcs.schash \leftarrow \text{SHA3\_512}(kcs.schash \parallel hmc)$ 
14:  $secr \leftarrow \text{KEM.Decapsulate}(cptb, kcs.deckey)$ 
15: if decapsulation fails then
16:    $cns.exflag \leftarrow \text{none}$ 
17:   return decapsulation_failure
18: end if
19:  // Derive symmetric keys and perform PSK ratchet, client view
20:  $prnd \leftarrow \text{cSHAKE512.XOF}(\text{input} = kcs.secl, \text{custom} = kcs.pssr, 2 \text{ blocks})$ 
21:  $tck_{C \rightarrow S}.key \leftarrow prnd[0..63]$ 
22:  $tck_{C \rightarrow S}.nonce \leftarrow prnd[64..95]$ 
23: Initialize  $cns.txcp$  as RCS with  $tck_{C \rightarrow S}$  in transmit mode
24:  $kcs.pssl \leftarrow \text{cSHAKE512}(kcs.pssl, tck_{C \rightarrow S}.key[0..31])$ 
25:  $prnd \leftarrow \text{cSHAKE512.XOF}(\text{input} = secr, \text{custom} = kcs.pssl, 2 \text{ blocks})$ 
26:  $tck_{S \rightarrow C}.key \leftarrow prnd[0..63]$ 
27:  $tck_{S \rightarrow C}.nonce \leftarrow prnd[64..95]$ 
28: Initialize  $cns.rxcp$  as RCS with  $tck_{S \rightarrow C}$  in receive mode
29:  $kcs.pssr \leftarrow \text{cSHAKE512}(kcs.pssr, tck_{S \rightarrow C}.key[0..31])$ 
30:  // Build establish request with encrypted transcript hash
31:  $mttype \leftarrow \text{establish\_request}$ 
32:  $milen \leftarrow \text{KEX\_ESTABLISH\_REQUEST\_MESSAGE\_SIZE}$ 
33:  $\text{HeaderCreate}(pkt_{out}, mttype, cns.txseq, milen)$ 
34:  $shdr' \leftarrow \text{SerializeHeader}(pkt_{out})$ 
35: Configure RCS associated data:  $\text{RCS.SetAD}(cns.txcp, shdr')$ 
36:  // Encrypt the raw transcript hash; the header is bound via the AEAD tag
37:  $\text{ciphertext} \leftarrow \text{RCS.Encrypt}(cns.txcp, kcs.schash)$ 
38: Write ciphertext into  $pkt_{out}.pmessage$ 
39:  // Advance the transcript with the outbound establish-request ciphertext
40:  $kcs.schash \leftarrow \text{SHA3\_512}(kcs.schash \parallel \text{ciphertext})$ 
41:  $cns.exflag \leftarrow \text{establish\_request}$ 
42: return none
```

---

**Server: Establish Response.** This function receives the Establish Request, decrypts it under the newly derived RCS receive context with the establish-request header as associated data, and compares the decrypted value against the current session transcript hash. If the values match, it advances its transcript by absorbing the inbound ciphertext, encrypts the updated transcript hash under the transmit RCS context, and returns it to the client. This step proves that both sides agree

on all asymmetric and symmetric handshake material.

---

**Algorithm 6** SERVERESTABLISHRESPONSE

---

**Require:** Server kex state  $kss$ , connection state  $cns$ , client packet  $pkt_{in}$  (establish request)

**Ensure:** Outgoing packet  $pkt_{out}$  (establish response) or error

```

1:  $shdr \leftarrow \text{SerializeHeader}(pkt_{in})$ 
2:  $\text{RCS.SetAD}(cns.rxcpr, shdr)$ 
3:  $hm \leftarrow \text{RCS.Decrypt}(cns.rxcpr, pkt_{in}.pmessage)$ 
4: if RCS authentication fails then
5:    $cns.exflag \leftarrow \text{none}$ 
6:   return decryption_failure
7: end if
8: if  $\text{VerifyEqual}(hm, kss.schash) \neq 0$  then
9:    $cns.exflag \leftarrow \text{none}$ 
10:  return verify_failure
11: end if
12:  $mtype \leftarrow \text{establish\_response}$ 
13:  $mten \leftarrow \text{KEX\_ESTABLISH\_RESPONSE\_MESSAGE\_SIZE}$ 
14:  $\text{HeaderCreate}(pkt_{out}, mtype, cns.txseq, mten)$ 
15:   // Advance the transcript with the inbound establish-request ciphertext
16:  $kss.schash \leftarrow \text{SHA3\_512}(kss.schash \parallel pkt_{in}.pmessage)$ 
17:  $shdr' \leftarrow \text{SerializeHeader}(pkt_{out})$ 
18:  $\text{RCS.SetAD}(cns.txcpr, shdr')$ 
19:   // Encrypt the updated transcript hash under the transmit cipher
20:  $\text{ciphertext} \leftarrow \text{RCS.Encrypt}(cns.txcpr, kss.schash)$ 
21: Write ciphertext into  $pkt_{out}.pmessage$ 
22:  $cns.exflag \leftarrow \text{session\_established}$ 
23: return none

```

---

**Client: Establish Verify.** This function processes the Establish Response and verifies that the decrypted transcript hash matches the client’s current transcript value (the value held after its own establish-request update). If the decryption and comparison succeed, the session is fully established and the tunnel keys, nonces, PSKs, and RCS contexts are active. This completes the handshake and transitions the client into the established tunnel state.

---

**Algorithm 7** CLIENTESTABLISHVERIFY

---

**Require:** Client kex state  $kcs$ , connection state  $cns$ , server packet  $pkt_{in}$  (establish response)

**Ensure:** Success or error

```
1:  $shdr \leftarrow \text{SerializeHeader}(pkt_{in})$ 
2:  $\text{RCS.SetAD}(cns.rxcpr, shdr)$ 
3:  $hm \leftarrow \text{RCS.Decrypt}(cns.rxcpr, pkt_{in}.pmessage)$ 
4: if RCS authentication fails then
5:   return decryption_failure
6: end if
7:   // Compare against the client transcript after its establish-request update
8: if  $\text{VerifyEqual}(hm, kcs.schash) \neq 0$  then
9:   return verify_failure
10: end if
11:  $cns.exflag \leftarrow \text{session\_established}$ 
12: return none
```

---

## 5 Security Definitions

This section defines the security goals for DKTP in terms of oracle based games between a challenger and an adversary. These games reflect the exact structure of the DKTP handshake and tunnel, including the transcript based binding mechanism, the directional shared secrets from the KEM, the mandatory directional pre-shared keys, and the RCS based authenticated encryption used for transport packets.

All definitions apply to protocol sessions created by calls to the **Send** oracle. A session is denoted by  $(P, \text{sid})$  where  $P$  is a party and  $\text{sid}$  is a locally unique session identifier.

### 5.1 Partnering

Two sessions  $(P, \text{sid})$  and  $(Q, \text{sid}')$  are partners if:

- each session identifies the other as its intended peer,
- both sessions hold the same rolling transcript hash  $\text{sch}$  after the establish phase,
- both sessions have verified the signatures on all handshake messages and the encrypted transcript hash comparison at establishment,
- both sessions complete without error and derive matching directional shared secrets and tunnel keys for their respective directions.

Partnering ensures that both parties agree on:

- configuration parameters,
- key identities,
- the complete exchange history as committed by the transcript hash,
- directional shared secrets  $\text{secl}$  and  $\text{secl}$ ,
- directional pre-shared keys after update,
- directional tunnel keys  $tckl$  and  $tckr$ .



## 5.2 Freshness

A session is fresh when its key material has not been trivially exposed. The exact conditions differ by game.

**Freshness for authentication.** A session is fresh for authentication if the long-term signature secret key of the peer being authenticated has not been revealed at any time prior to completion of the session.

**Freshness for handshake confidentiality.** A completed session  $(P, \text{sid})$  is fresh if:

- neither directional tunnel key  $tckl$  or  $tckr$  for this session has been revealed,
- the ephemeral KEM private keys for this session have not been revealed,
- the adversary has not modified or injected a different transcript into the session,
- the long-term signing key of the partner was not corrupted prior to completion.

**Freshness for transport confidentiality and integrity.** A transport session is fresh if:

- neither tunnel key  $tckl$  nor  $tckr$  has been revealed,
- the sequence number for the ciphertext has not been used before,
- the corresponding PSK for the direction has not been revealed (which the model forbids entirely).

**Freshness for ratchet and PCS.** A session is fresh for the ratchet or PCS games if:

- the adversary may corrupt long-term keys prior to the handshake but
- must not reveal the new directional tunnel keys generated by the handshake for which the Test query is issued.

## 5.3 Authentication Game

Authentication captures the requirement that no adversary may cause a party to complete a handshake with a peer that did not participate.

**Game structure.**

- The adversary interacts with the protocol through **Send**, **Corrupt**, and **RevealEPH**.
- The adversary attempts to cause an honest session  $(P, \text{sid})$  to complete and accept peer identity  $Q$ .
- The adversary succeeds if no session at  $Q$  partners with  $(P, \text{sid})$ .

The adversary's advantage is:

$$\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) = \Pr[\mathcal{A} \text{ wins the authentication game}].$$

This captures impersonation resistance and reliance on the transcript and signature mechanisms.

## 5.4 Handshake Confidentiality Game

This game models confidentiality of the initial tunnel keys derived during the handshake.

### Game structure.

- The adversary runs the protocol and chooses a fresh completed session  $(P, \text{sid})$ .
- The adversary queries  $\mathbf{Test}(P, \text{sid})$ .
- The challenger flips a bit  $b$ .
- If  $b = 0$ , the challenger returns the real directional tunnel key.
- If  $b = 1$ , it returns a random key of the same length.
- The adversary continues interacting with all oracles except  $\mathbf{RevealTunnelKey}$  for that session.
- The adversary outputs a guess  $b'$ .

The advantage is:

$$\text{Adv}_{\mathcal{A}}^{\text{conf}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

## 5.5 Ciphertext Integrity Game

This game captures the integrity of RCS encrypted transport packets.

### Game structure.

- The adversary may send arbitrary ciphertexts to honest receivers.
- The adversary may observe encryptions produced by honest parties through  $\mathbf{Send}$ .
- The adversary succeeds if it causes an honest receiver to accept a ciphertext that was not produced by the sender under the same sequence number and header.

The advantage is:

$$\text{Adv}_{\mathcal{A}}^{\text{int}}(\lambda) = \Pr[\mathcal{A} \text{ produces a valid forgery}].$$

## 5.6 Replay and Reordering Game

This models protection against replay and out of order delivery.

### Game structure.

- The adversary may deliver any encrypted packet with any header.
- Let  $\text{seq}_{\max}$  denote the highest accepted sequence number in that direction.
- The adversary wins if it causes acceptance of a ciphertext with sequence number  $\text{seq} \leq \text{seq}_{\max}$ .

The advantage is:

$$\text{Adv}_{\mathcal{A}}^{\text{replay}}(\lambda) = \Pr[\mathcal{A} \text{ replays or reorders a packet successfully}].$$

## 5.7 Forward Secrecy Game

Forward secrecy ensures that compromise of long-term secret keys after the handshake does not reveal past symmetric tunnel keys.

**Game structure.**

- The adversary interacts with the protocol normally.
- The adversary selects a completed session  $(P, \text{sid})$ .
- After the handshake completes, the adversary obtains the long-term signing keys of both parties through **Corrupt**.
- The adversary queries **Test** $(P, \text{sid})$  on the tunnel key.

Forward secrecy holds if:

$$\text{Adv}_{\mathcal{A}}^{\text{fs}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|$$

is negligible.

**5.8 Post Compromise Security Game**

PCS models recovery from compromise when a new handshake occurs.

**Game structure.**

- The adversary learns long-term signature keys or previous PSKs through **Corrupt** or prior compromise.
- A new handshake is executed.
- The adversary queries **Test** on the tunnel key derived from the new handshake.
- The adversary wins by distinguishing real from random.

Directional PSK updates and new KEM shared secrets support recovery.

**5.9 Ratchet Game**

The ratchet game models the one-way evolution of tunnel keys and PSKs between handshake epochs.

**Game structure.**

- The adversary may reveal tunnel keys or ephemeral secrets from any ratchet step  $i$ .
- The adversary then requests **Test** on the tunnel key at step  $i + 1$ .
- The challenger returns either the real or a random key.
- The adversary outputs a guess.

The advantage is:

$$\text{Adv}_{\mathcal{A}}^{\text{ratchet}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

A successful violation would contradict the pseudo-randomness of the KDF and the independence of the new KEM shared secrets.

## 6 Security Proofs

This section presents formal reduction based proofs for the stated DKTP security goals. The proofs reflect the DKTP protocol structure, including transcript binding, directional pre-shared key state, directional KEM shared secrets, and 512-bit tunnel keys.

### 6.1 Transcript Integrity Lemma (Including PSK Inputs)

**Lemma 1** (Transcript Integrity). *Consider an honest initiator session and an honest responder session that complete the DKTP handshake. Let  $\text{Adv}^{\text{tint}}$  denote the probability that the two sessions accept while their ordered transcripts of  $M_1, \dots, M_6$  differ in any field. Let  $q_s$  be the total number of **Send** queries,  $n_u$  the number of long-term verification keys,  $q_H$  the number of distinct Keccak evaluations made by the experiment and the adversary, and  $\ell$  the SHA3-512 output length in bits ( $\ell = 512$ ). Then*

$$\text{Adv}^{\text{tint}} \leq n_u \cdot \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}} + \text{Adv}_{\text{RCS}}^{\text{INT-CTXT}} + \frac{q_H^2}{2^\ell}.$$

*Proof.* The transcript hash is the Keccak chain

$$\text{sch}_0 = \text{H}(\text{hm}_1), \quad \text{sch}_i = \text{H}(\text{sch}_{i-1} \parallel h_i) \quad (i \geq 1),$$

where  $\text{hm}_1 = \text{H}(\text{sph}_1 \parallel \text{kid} \parallel \text{cfg})$  and each  $h_i$  is the SHA3-512 digest of the authenticated payload of stage  $i$  together with its serialized header. We bound the event by a sequence of games.

*Game  $G_0$ .* The real experiment.  $\text{Adv}_{G_0} = \text{Adv}^{\text{tint}}$ .

*Game  $G_1$  (collision freeness).*  $G_1$  aborts if any two distinct Keccak inputs queried during the experiment collide under SHA3-512. Over  $q_H$  evaluations the birthday bound gives  $|\Pr[G_1] - \Pr[G_0]| \leq q_H^2/2^\ell$ . Conditioned on  $\neg \text{abort}$ , the maps  $x \mapsto \text{H}(x)$  used to form  $\text{hm}_1$ , each  $h_i$ , and each chaining step are injective on the queried domain. Hence two sessions reach equal  $\text{sch}$  values only if they absorbed identical inputs at every stage; equal final transcript hashes therefore imply identical ordered transcripts of  $M_1, \dots, M_4$  together with identical establish-request ciphertext absorbed at stage 5.

*Game  $G_2$  (signature unforgeability).* Messages  $M_1, M_3$  are signed by the initiator and  $M_2, M_4$  by the responder, in each case over  $\text{H}(\text{sph} \parallel \text{payload})$ .  $G_2$  aborts if any honest session accepts a signed message  $M_i$  ( $i \leq 4$ ) whose (header, payload) pair was never produced by the purported honest sender. A reduction  $\mathcal{B}_{\text{sig}}$  guesses the owner of the abused key among the  $n_u$  verification keys, embeds its EUF-CMA challenge key, answers signing through the signing oracle, and outputs the accepted (message, signature) pair as a forgery; the guessed message was never signed by the honest party, so it is fresh. Thus  $|\Pr[G_2] - \Pr[G_1]| \leq n_u \cdot \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}$ . In  $G_2$  every accepted  $M_1$ – $M_4$  field is exactly the value emitted by the honest peer, so the chained inputs through stage 4 are pinned to the honest transcript.

*Game  $G_3$  (establish-phase ciphertext integrity).* After stage 4 the two sessions hold identical  $\text{sch}$  (by  $G_1, G_2$ ). At establish, the initiator sends  $\text{cpt}_5 \leftarrow \text{RCS}.\text{Enc}_{\text{tck}_{I \rightarrow R}}(\text{sch}; AD = \text{sph}_5)$  and both absorb  $\text{cpt}_5$  into  $\text{sch}$ ; the responder replies with  $\text{cpt}_6 \leftarrow \text{RCS}.\text{Enc}_{\text{tck}_{R \rightarrow I}}(\text{sch}; AD = \text{sph}_6)$ .  $G_3$  aborts if a receiver accepts an establish ciphertext under associated data  $\text{sph}$  that the legitimate sender never produced for that (key,  $AD$ ) pair. A reduction  $\mathcal{B}_{\text{rCS}}$  relays all establish-phase encryptions to its INT-CTXT challenger and submits the accepted-but-unproduced ciphertext as its forgery, so  $|\Pr[G_3] - \Pr[G_2]| \leq \text{Adv}_{\text{RCS}}^{\text{INT-CTXT}}$ . In  $G_3$  the establish-request ciphertext absorbed at stage 5 is likewise pinned to the honest value.

In  $G_3$  both sessions accept only if their ordered transcripts of  $M_1, \dots, M_6$  are field-for-field identical, so the integrity-violation event has probability 0. Collecting the three transitions yields the stated bound.  $\square$

**Remark.** The header binding at every stage is the AEAD/signature associated data, not a separate domain-separated digest: handshake messages bind `sph` inside the *signed* hash, and establish messages bind `sph` as RCS associated data. The fixed ratchet domain string is not part of any establish confirmation.

## 6.2 Authentication Proof

**Theorem 1** (Mutual Authentication). *Let  $n_u$  be the number of honest long-term verification keys and  $q_s$  the number of **Send** queries. If  $\mathcal{A}$  does not corrupt the long-term signing key of the peer it impersonates before the target session completes, then*

$$\text{Adv}_{\mathcal{A}}^{\text{auth}}(\lambda) \leq n_u \cdot \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}(\lambda) + \text{Adv}^{\text{tint}}(\lambda),$$

where  $\text{Adv}^{\text{tint}}$  is the transcript-integrity bound of Lemma 1. In particular DKTP provides mutual authentication: a completed honest session has a partnered session at the claimed peer that agrees on the full transcript.

*Proof.* We prove entity authentication in both directions and then upgrade to matching conversations.

*Responder-to-initiator.* Suppose an honest initiator session at  $P$  completes and names peer  $Q$ , but no session of  $Q$  ever ran with  $P$ . The initiator accepts only after verifying  $\sigma_2$  on  $\text{H}(\text{sph}_2 \parallel \text{ek}_R)$  and  $\sigma_4$  on  $\text{H}(\text{sph}_4 \parallel c_{ptb})$  under  $vk_Q$ , and after the establish-verify comparison. Reduction  $\mathcal{B}$  samples  $j \leftarrow \{1, \dots, n_u\}$ , plants the EUF-CMA challenge key as the  $j$ -th verification key, and simulates all parties, answering signatures for the challenge identity through its signing oracle and generating all other keys itself. If  $\mathcal{A}$ 's target peer is the  $j$ -th identity (probability  $1/n_u$  conditioned on a win) and  $Q$  never participated, then no honest session of  $Q$  ever requested a signature on  $\text{H}(\text{sph}_2 \parallel \text{ek}_R)$  for this header;  $\mathcal{B}$  outputs  $(\text{H}(\text{sph}_2 \parallel \text{ek}_R), \sigma_2)$  as a forgery. Because `sph` contains the per-session sequence number and UTC timestamp, replays of a genuine  $Q$ -signature fall outside the validity window or fail the sequence predicate, so the forwarded message is fresh.

*Initiator-to-responder.* Symmetrically, an honest responder accepts only after verifying  $\sigma_1$  over  $\text{H}(\text{sph}_1 \parallel \text{kid} \parallel \text{cfg})$  and  $\sigma_3$  over  $\text{H}(\text{sph}_3 \parallel c_{pta} \parallel \text{ek}_I)$ , and after the establish-request decryption-and-compare. The same reduction (now targeting the initiator identity) converts a non-participating-initiator acceptance into an EUF-CMA forgery on  $\sigma_1$ .

Either direction therefore costs at most  $n_u \cdot \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}}$ ; the two reductions share the same key-guess and are mutually exclusive on a single win, so the union does not double the factor.

*From entity authentication to matching conversations.* Entity authentication guarantees a genuine partner session at  $Q$ ; Lemma 1 guarantees that two accepting honest sessions agree field-for-field on  $M_1, \dots, M_6$ , hence on configuration, identities, ephemeral keys, KEM ciphertexts, and the order of messages. Adding the two contributions gives the stated bound. Since the establish phase is reached only after both directional signatures verify and the encrypted transcript comparison succeeds, acceptance implies a partner, which is the definition of mutual authentication.  $\square$

## 6.3 Key Indistinguishability Proof with PSK

**Theorem 2** (Directional Key Indistinguishability). *For a fresh completed session (Section 5.2), the advantage of any adversary in the handshake-confidentiality game against a tested directional tunnel key is bounded by*

$$\text{Adv}_{\mathcal{A}}^{\text{conf}} \leq 2 \text{Adv}_{\text{KEM}}^{\text{IND-CCA}} + 2 \text{Adv}_{\text{CSHAKE}}^{\text{PRF}} + \text{Adv}^{\text{tint}},$$

where  $\text{Adv}^{\text{tint}}$  is the transcript-integrity bound of Lemma 1.

*Proof.* By Lemma 1 and Theorem 1 we may, at the cost of the additive term  $\text{Adv}^{\text{tint}}$ , restrict attention to a tested session that has an honest partner, so the KEM public keys  $\text{ek}_R, \text{ek}_I$  used in derivation were generated by honest parties and the corresponding ciphertexts were not adversarially crafted. We proceed by a hybrid argument; freshness forbids **RevealEPH** and **RevealTunnelKey** on the tested session, so the relevant ephemeral KEM private key is unknown to  $\mathcal{A}$ .

$G_0$ . The handshake-confidentiality game with the real key schedule.

$G_1$  (*KEM secret of the tested direction*). Replace the KEM shared secret that keys the tested direction (*secl* for the  $I \rightarrow R$  key, *secl* for the  $R \rightarrow I$  key) with an independent uniform value. A distinguisher yields an IND-CCA distinguisher that embeds the challenge public key as the relevant  $\text{ek}$  and the challenge ciphertext as the corresponding handshake ciphertext, answering any other decapsulation through the (CCA) decapsulation oracle; ciphertexts differ from the challenge by transcript integrity. Hence  $|\Pr[G_1] - \Pr[G_0]| \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}$ .

$G_2$  (*channel KDF as a PRF*). The tested key is  $tck \parallel n \parallel \dots = \text{cSHAKE512}(X = s, N = \text{label}, S = p)$ , where after  $G_1$  the keying input  $s$  is uniform and independent. Model cSHAKE-512 with fixed domain label as a PRF keyed by its secret input. Replace the 96-byte output block by a uniform string. Then  $|\Pr[G_2] - \Pr[G_1]| \leq \text{Adv}_{\text{cSHAKE}}^{\text{PRF}}$ , and in  $G_2$  the tested  $tck$  is uniform and independent of the challenge bit, so the adversary's advantage is 0.

For the  $R \rightarrow I$  direction the keying material is *secl* and the customization is the updated PSK  $pssl'$ . The two hybrids above apply directly with  $s = \text{secl}$ ; the dependence on  $pssl'$  does not weaken the bound because  $pssl'$  enters only as the (public-domain) customization input and the PRF is keyed by the fresh *secl*. Summing the two directions (each costing one KEM and one PRF step) gives the stated bound.  $\square$

**Dual-key (KEM-or-PSK) guarantee.** The bound above keys the PRF on the KEM secret. DKTP additionally absorbs the directional PSK as the cSHAKE customization string  $S$ . In the Keccak sponge, cSHAKE initializes its state by absorbing  $\text{bytepad}(\text{encode}(N) \parallel \text{encode}(S))$  before the message input  $X$ ; a secret, high-entropy  $S$  therefore contributes secret entropy to the initial sponge state. Consequently the channel output is pseudorandom whenever *at least one* of the two absorbed secrets is unknown to the adversary: if the KEM secret  $X$  is fresh, pseudorandomness follows from keying on  $X$  (the theorem above); if instead  $X$  is exposed but the PSK  $S$  retains  $\kappa$  bits of secret entropy, pseudorandomness follows from keying the sponge on  $S$ , with distinguishing advantage bounded by the sponge PRF term plus  $q^2/2^c$  for sponge capacity  $c$  and  $q$  Keccak calls. This is the formal content of the “two disparate systems must be broken” design claim: an adversary must defeat both the KEM *and* the PSK confidentiality for a given direction to recover that channel key.

## 6.4 Ciphertext Integrity Proof

**Theorem 3** (Ciphertext Integrity). *Let  $q_e$  and  $q_d$  be the number of transport encryption and decryption (delivery) queries in a direction. If RCS is INT-CTXT secure, then the probability that an honest receiver accepts a transport packet not produced by its partner under the same key, header, and stream position is at most*

$$\text{Adv}_{\mathcal{A}}^{\text{int}} \leq 2 \text{Adv}_{\mathcal{A}}^{\text{conf}} + 2 \text{Adv}_{\text{RCS}}^{\text{INT-CTXT}}(q_e, q_d),$$

*the factor two covering the two directions and the  $\text{Adv}^{\text{conf}}$  term replacing the real, derived tunnel keys by independent random keys via Theorem 2.*

*Proof.* First apply Theorem 2 in each direction to replace the directional tunnel keys  $tck_{I \rightarrow R}, tck_{R \rightarrow I}$  with independent uniform keys; this costs  $2 \text{Adv}^{\text{conf}}$  and makes each transport channel an instance

of RCS keyed by a fresh random key. A reduction  $\mathcal{B}$  then plays the INT-CTXT game for that key: it answers honest encryptions through the RCS encryption oracle (serializing the 21-byte header as associated data) and submits any delivered packet that decrypts-and-verifies but was never produced by the partner for that (key,  $AD$ , position) triple as its forgery. Because the entire header — flag, length, sequence, and timestamp — is authenticated as associated data, any change to those fields produces a fresh ( $AD$ ) and hence a valid INT-CTXT forgery. Summing the two directions yields the bound.  $\square$

## 6.5 Replay and Ordering Proof

**Theorem 4** (Replay and Reordering Rejection). *The probability that an honest receiver accepts a replayed, stale, skipped, or reordered transport packet is at most  $\text{Adv}_{\mathcal{A}}^{\text{int}}$  (Theorem 3).*

*Proof.* The receiver maintains a per-direction counter and accepts a packet only if  $\text{seq} = \text{rxseq} + 1$  (and the timestamp lies within the validity window), then increments  $\text{rxseq}$ . A genuine prior ciphertext re-delivered unchanged carries its original  $\text{seq} \leq \text{rxseq}$  and is rejected by the predicate with probability 1; a skipped or reordered ciphertext has  $\text{seq} \neq \text{rxseq} + 1$  and is likewise rejected. The only way to obtain acceptance out of order is to present a ciphertext that authenticates under the current key with a header (hence  $AD$ ) carrying  $\text{seq} = \text{rxseq} + 1$  that the partner never emitted at that stream position; by Theorem 3 this happens with probability at most  $\text{Adv}_{\mathcal{A}}^{\text{int}}$ . The timestamp field, also authenticated as  $AD$ , additionally bounds the window in which any captured packet could be presented.  $\square$

## 6.6 Forward Secrecy

**Theorem 5** (Forward Secrecy for Completed Sessions). *Let  $\mathcal{A}$  complete the handshake of a tested session and only afterward corrupt the long-term signing keys of both parties. Then*

$$\text{Adv}_{\mathcal{A}}^{\text{fs}} \leq \text{Adv}_{\mathcal{A}}^{\text{conf}} = 2 \text{Adv}_{\text{KEM}}^{\text{IND-CCA}} + 2 \text{Adv}_{\text{cSHAKE}}^{\text{PRF}} + \text{Adv}^{\text{tint}}.$$

*Proof.* The signing keys  $sk_I, sk_R$  authenticate handshake messages but are never inputs to the key schedule: the tunnel keys are functions only of the KEM shared secrets  $\text{secl}, \text{secl}$ , the directional PSKs, and the fixed domain labels. Corrupting  $sk_I, sk_R$  after completion therefore gives  $\mathcal{A}$  no additional information about the tested key, so the post-completion **Corrupt** queries can be simulated by the reduction of Theorem 2 without affecting the tested challenge. The bound is exactly  $\text{Adv}_{\mathcal{A}}^{\text{conf}}$ .

*Erasure scope.* The forward-secrecy claim is for *completed* sessions. The reference implementation erases the responder’s transient shared secrets during exchange-response processing, and erases the initiator’s first shared secret and ephemeral decapsulation key when key-exchange state is reset at handshake completion. Within this finite window an explicit **RevealEPH** would trivially break the tested key, which is why freshness forbids it for the tested session; the window does not affect sessions whose ephemeral material has been erased.  $\square$

## 6.7 Post Compromise Security

**Theorem 6** (Post-Compromise Recovery). *Suppose at some epoch the adversary has learned a directional tunnel key and the corresponding prior PSK state, but a subsequent authenticated handshake or asymmetric-ratchet exchange introduces a fresh KEM shared secret  $\text{ssec}$  whose ephemeral private key is not revealed. Then the post-recovery directional key is indistinguishable*

from random with advantage at most

$$\text{Adv}_A^{\text{PCS}} \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA}} + \text{Adv}_{\text{cSHAKE}}^{\text{PRF}} + \text{Adv}_A^{\text{int}},$$

where the  $\text{Adv}^{\text{int}}$  term covers the AEAD-protected, signed ratchet messages that carry the fresh KEM material.

*Proof.* Recovery re-keys the direction from

$$\text{prnd} \leftarrow \text{cSHAKE512}(X = \text{ssec}, N = \text{QRCS:PDEF:DKTP1A}, S = \text{pss}).$$

Even granting the adversary full knowledge of the pre-recovery  $\text{pss}$ , the keying input  $\text{ssec}$  is a fresh KEM secret. By IND-CCA security,  $\text{ssec}$  can be replaced by a uniform value at cost  $\text{Adv}_{\text{KEM}}^{\text{IND-CCA}}$ . The cSHAKE PRF step then makes  $\text{prnd}$  uniform and independent of all pre-recovery state at cost  $\text{Adv}_{\text{cSHAKE}}^{\text{PRF}}$ . The adversary cannot substitute its own ratchet material in an uncompromised established session because the ratchet request and response are carried inside the authenticated transport and include signature checks; forged material is covered by the authenticated-channel integrity term. The subsequent update

$$\text{pss}' \leftarrow \text{cSHAKE512}(X = \text{prnd}[0..31], N = \epsilon, S = \text{pss})$$

carries the fresh entropy into persistent state, so the compromised pre-recovery PSK does not determine  $\text{pss}'$ .  $\square$

## 6.8 Ratchet Security

**Theorem 7** (Ratchet Key Independence). *For an asymmetric-ratchet step whose KEM ephemeral key is unrevealed, the new directional tunnel key is indistinguishable from random given all prior transport key material, with advantage at most  $\text{Adv}_{\text{KEM}}^{\text{IND-CCA}} + \text{Adv}_{\text{cSHAKE}}^{\text{PRF}} + \text{Adv}_A^{\text{int}}$ .*

*Proof.* The ratchet derivation is

$$\begin{aligned} \text{prnd} &\leftarrow \text{cSHAKE512}(X = \text{ssec}, N = \text{QRCS:PDEF:DKTP1A}, S = \text{pss}), \\ \text{pss}' &\leftarrow \text{cSHAKE512}(X = \text{prnd}[0..31], N = \epsilon, S = \text{pss}). \end{aligned}$$

The dedicated domain string `QRCS:PDEF:DKTP1A` separates this derivation from the handshake KDF (distinct domain inputs  $\Rightarrow$  separated sponge invocations under the domain-separation assumption), so prior handshake outputs give no leverage. The same two hybrids as in Theorem 2 (replace  $\text{ssec}$  by uniform via IND-CCA, then the cSHAKE block by uniform via the PRF assumption) make the new key uniform and independent of all earlier material; the in-tunnel transport of the ratchet messages is signed and AEAD-protected, contributing the  $\text{Adv}^{\text{int}}$  term against active injection. Finally the PSK update prevents reuse of the superseded directional state after a successful ratchet.  $\square$

## 6.9 Composition Corollary

The DKTP construction combines signatures, KEM outputs, transcript hashing, cSHAKE-based KDFs, PSK state updates, and RCS authenticated encryption in a tightly coupled secure-channel design. Its security follows by hybrid reasoning over the authenticated handshake, the KEM-derived key schedule, and the AEAD-protected transport layer. The composition depends on the transcript hash, client-perspective channel labels, serialized-header binding, and sequential PSK update order.

Under the stated assumptions on the selected KEM, signature scheme, SHA3/cSHAKE functions, PSK storage, and RCS AEAD, DKTP achieves mutual authentication, key indistinguishability, ciphertext integrity, strict replay rejection, post-completion forward secrecy, and recovery after fresh KEM ratcheting.



## 7 Cryptanalysis

This section provides a detailed adversarial analysis of DKTP viewed from the perspective of a capable network level attacker. The goal is to identify all meaningful attack surfaces, characterize the power of adversaries allowed by the formal model, explain why attacks fail under the assumptions on the cryptographic primitives, and quantify the contribution of pre-shared keys, transcripts, and ratcheting to the overall security picture. Every vector considered here is derived from or explicitly addressed in the DKTP specification and is evaluated against the formal reductions proven earlier.

### 7.1 Attack Surfaces in DKTP

DKTP exposes several potential attack surfaces that a network attacker may attempt to exploit. These can be grouped according to the stage of the protocol.

#### Handshake surfaces.

- Modification of fields in Connect Request or Connect Response.
- Injection of false ephemeral KEM public keys.
- Replacement of KEM ciphertexts.
- Replay of old handshake messages.
- Substitution or replay of handshake messages across sessions.

The signature on each message hash (bound to the packet header including sequence number and timestamp) prevents impersonation. The rolling transcript hash binds  $M_1$  through  $M_4$  to the correct session configuration, identity, and complete exchange history. The encrypted transcript-hash comparison in the establish phase ( $M_5$ ,  $M_6$ , and the initiator's establish-verify) prevents manipulation of any intermediate handshake state.

#### Key schedule surfaces.

- Attempt to compute directional tunnel keys without both the KEM shared secret and the correct PSK.
- Attempt to derive the next PSK value without access to the first 32 bytes of the corresponding tunnel key.
- Attempt to exploit cross-channel key relationships.

These fail because the cSHAKE-512 KDF treats each direction separately with distinct channel identity labels, and uses mandatory directional PSKs that are updated inline during derivation.

#### Transport surfaces.

- Attempting to forge RCS ciphertexts.
- Attempting replay attacks by resending old packets.
- Attempting out of order delivery.
- Attempting to guess the header or tamper with associated data fields.

Transport packets use RCS authenticated encryption with associated data that includes the entire 21 byte header. RCS nonces do not repeat within a direction because each direction is

initialized once with a distinct KDF-derived 32-byte initial nonce and the RCS internal counter advances monotonically as data is processed. The packet sequence number is an authenticated header field used for ordering and replay rejection; it is not supplied to RCS as nonce material.

#### **Post compromise surfaces.**

- Attacks following compromise of static keys.
- Attacks following compromise of previous tunnel keys.
- Attempts to exploit downgraded configuration values.

A new handshake with fresh uncompromised KEM secrets can reestablish channel-key secrecy even if prior tunnel state was exposed, provided peer authentication, endpoint integrity, erasure, and persisted-state update assumptions hold.

## **7.2 Adversarial Capabilities and Limitations**

The adversary controls the network completely. It may reorder, inject, replay, delay, or drop messages. It may interact through the following oracles:

- **Send**: deliver handshake or transport messages and obtain the peer's response.
- **CorruptSig**: reveal long-term signing keys.
- **RevealEPH**: reveal ephemeral KEM private keys for a chosen session.
- **RevealPSK**: reveal persisted peering secrets `pssl` and/or `pssr` at a chosen time.
- **RevealTunnelKey**: reveal directional tunnel keys for a chosen established session.
- **Test**: challenge the indistinguishability of session keys for a fresh session.

Adversarial limitations stem from the cryptographic assumptions:

- it cannot compute `secl` or `secl` for a fresh session without the corresponding ephemeral KEM private key,
- it cannot forge valid handshake signatures over the authenticated transcript hash and serialized header fields,
- it cannot compute DKTP KDF outputs without the required secret inputs for that derivation,
- it cannot modify or forge RCS ciphertexts except with negligible probability under the AEAD security bound,
- it cannot induce nonce reuse within a direction when sequence numbers are enforced and authenticated as associated data.

## **7.3 Effects of PSK Compromise**

This subsection analyzes compromise of the pre-shared secrets stored in the peering key structures:

`pssl` (local pre-shared secret),      `pssr` (remote pre-shared secret).

**Effect on channel key derivation.** DKTP derives channel keys as:

$$(tckl, nl) \leftarrow \text{KDF}_{\text{DKTP}}(\text{secl}, \text{pssr}), \quad (tckr, nr) \leftarrow \text{KDF}_{\text{DKTP}}(\text{secl}, \text{pssl}).$$

If an adversary compromises  $\text{pssl}$  and/or  $\text{pssr}$ , the adversary removes one secret input from the KDF. Security then depends primarily on the confidentiality of the corresponding KEM shared secret for that direction.

**PSK update and persistence.** During key derivation, DKTP updates the opposing pre-shared secrets:

$$\text{pssl} \leftarrow \text{cSHAKE512}(\text{pssl}, \perp, tckl[0:32]), \quad \text{pssr} \leftarrow \text{cSHAKE512}(\text{pssr}, \perp, tckr[0:32]).$$

Therefore, compromise of an old pre-shared secret does not automatically imply compromise of its updated successor after a fresh handshake in which the updated state is computed and persisted.

**Interaction with long-term key compromise.** Compromise of long-term signature keys enables impersonation at the message authentication layer, but does not by itself reconstruct past channel keys without breaking the KEM secrecy for the corresponding sessions and directions.

This analysis assumes the adversary may compromise stored pre-shared secrets at arbitrary times; the security games should interpret such compromise as removal of the corresponding secret KDF input for sessions after the compromise event.

## 7.4 KCI and UKS Resistance

**Key compromise impersonation resistance.** If the adversary compromises  $sk_I$ , it cannot impersonate  $R$  to  $I$  because impersonation requires producing valid signatures under  $sk_R$  for messages whose hashes are bound to fresh packet headers. The rolling transcript hash binds the configuration, identities, and all exchanged keying material, so the attacker cannot introduce a false identity or substitute any handshake content without detection at the establish phase.

**Unknown key share resistance.** The attacker cannot cause  $I$  and  $R$  to complete handshakes that share a key but have different peer identities. The transcript hash accumulates all authenticated handshake material including the signed ephemeral keys, so the peers must independently arrive at an identical  $\text{sch}$  to pass the establish-phase comparison. A manipulated handshake would violate the transcript integrity lemma.

## 7.5 Full Analysis of AEAD and Nonce Construction

**Nonce derivation.** DKTP derives a 32-byte initial nonce for each RCS direction from bytes 64 through 95 of the cSHAKE-512 output block. RCS then advances its internal counter as data is processed. DKTP packet sequence numbers are not encoded as nonces and are not supplied to RCS as nonce material.

**Sequence and nonce separation.** The DKTP sequence number provides replay and ordering protection. The RCS nonce provides per-cipher-state uniqueness. These mechanisms are separate. A packet sequence value is authenticated as part of the serialized packet header, while the RCS nonce evolves inside the cipher state after initialization.

**Associated data.** The serialized packet header is authenticated as RCS associated data. Its order is flag, message length, sequence number, and UTC timestamp, with all multi-byte fields encoded little-endian.

**Integrity consequences.** A successful packet forgery would require producing a ciphertext and tag that authenticate under the current RCS state and serialized header. A replayed packet fails the receiver sequence predicate unless it is presented at exactly the expected sequence position, and even then its timestamp and state-dependent authentication must verify.

## 7.6 Quantum Random Oracle Considerations

The KDF and transcript hash rely on Keccak-based constructions. Keccak is modeled as a quantum random oracle in the formal analysis. Under this model:

- adversarial superposition queries do not reveal input structure,
- the domain separation labels prevent cross protocol interference,
- KDF outputs are pseudo-random even under quantum adversaries,
- collisions on the transcript hash remain negligible.

The security of directional tunnel keys depends on the pseudo-randomness of cSHAKE-512 outputs given the KEM shared secret and the corresponding directional PSK. Both inputs remain unknown to the adversary in fresh sessions.

## 7.7 Summary of Adversarial Bounds

Combining all reductions and attack surface analyses, writing  $\text{Adv}^{\text{tint}} \leq n_u \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}} + \text{Adv}_{\text{RCS}}^{\text{INT-CTXT}} + q_H^2/2^{512}$  for the transcript-integrity term of Lemma 1:

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{auth}} &\leq n_u \text{Adv}_{\text{SIG}}^{\text{EUF-CMA}} + \text{Adv}^{\text{tint}}, \\ \text{Adv}_{\mathcal{A}}^{\text{conf}} &\leq 2 \text{Adv}_{\text{KEM}}^{\text{IND-CCA}} + 2 \text{Adv}_{\text{cSHAKE}}^{\text{PRF}} + \text{Adv}^{\text{tint}}, \\ \text{Adv}_{\mathcal{A}}^{\text{int}} &\leq 2 \text{Adv}_{\mathcal{A}}^{\text{conf}} + 2 \text{Adv}_{\text{RCS}}^{\text{INT-CTXT}}, \quad \text{Adv}_{\mathcal{A}}^{\text{replay}} \leq \text{Adv}_{\mathcal{A}}^{\text{int}}, \\ \text{Adv}_{\mathcal{A}}^{\text{fs}} &\leq \text{Adv}_{\mathcal{A}}^{\text{conf}}, \quad \text{Adv}_{\mathcal{A}}^{\text{PCS}}, \text{Adv}_{\mathcal{A}}^{\text{ratchet}} \leq \text{Adv}_{\text{KEM}}^{\text{IND-CCA}} + \text{Adv}_{\text{cSHAKE}}^{\text{PRF}} + \text{Adv}_{\mathcal{A}}^{\text{int}}. \end{aligned}$$

Directional 512-bit tunnel keys provide a symmetric security level far beyond NIST Category 5. Even quantum adversaries gain no meaningful advantage due to the structure of the KDF, the mandatory use of separate PSKs, and the use of independent KEM shared secrets.

## 8 Implementation Conformance

This section maps the DKTP reference implementation to the formal model. The mapping covers the source files that implement the handshake, key schedule, KDF inputs, packet layout, and ratchet update.

### 8.1 Mapping Between Model and Code

The handshake logic in the implementation matches the formal protocol description.

**Message M1.** The Connect Request includes the initiator key identity and configuration string. The message hash is computed from the serialized packet header, key identity, and configuration string, and is signed under the initiator’s long-term signature key. The transcript hash is initialized from this message hash. Serialization order matches the formal definition.

**Message M2.** The Connect Response contains the responder ephemeral KEM public key and a signature covering the hash of the outbound packet header and encapsulation key. The responder initializes its transcript hash from the verified M1 message hash and updates it with the encapsulation key hash, exactly as described in the specification.

**Message M3.** The Exchange Request includes the KEM ciphertext, the initiator’s ephemeral encapsulation key, and a signature over their hash bound to the packet header. The transcript hash is updated with the verified and outgoing message hashes.

**Message M4.** The Exchange Response contains the responder’s KEM ciphertext and a signature over its hash bound to the packet header. The responder derives directional tunnel keys via cSHAKE-512, updates PSKs inline, and initializes cipher instances. The transcript hash is updated with the signed ciphertext hash.

**Key schedule.** The implementation derives directional tunnel keys using cSHAKE-512 with the KEM shared secret as the key input, the per-channel identity label as the customization parameter, and the corresponding PSK as the data input:

$$(tckl, nl) \leftarrow \text{cSHAKE512}(\text{key}=\text{secl}, \text{custom}=\text{"DKTP:TXKEY:V1"}, \text{data}=\text{pssr}).$$

PSKs are updated inline using the first 32 bytes of the derived key material. The same domain separation labels appear in the implementation. The 512-bit tunnel keys and initial nonces are derived exactly as in the formal specification.

**Transport phase.** The header fields (flag, message length, sequence, timestamp) appear in the exact order and with the exact sizes listed in the specification. The header is passed to RCS as authenticated associated data. The ciphertext is generated using the appropriate directional tunnel key and nonce. The receiver performs the exact same RCS call and checks that the header is authenticated.

**State machine.** The implementation follows the same state transitions as the formal model: Idle, Handshake, and Established. The optional asymmetric ratchet is processed inside the Established state. Messages out of order cause immediate failure.

## 8.2 Algorithm Agility in dktp.h

Algorithm agility is encoded directly in the implementation through compile-time configuration macros in `dktp.h`. The implementation supports multiple KEM and signature schemes through the following mechanisms.

**Configurable KEM.** The KEM key generation and decapsulation functions are accessed through wrappers whose implementation depends on the selected KEM family. The formal model treats the KEM abstractly, and the implementation reflects this abstraction correctly.

**Configurable signature scheme.** The signature functions are selected by configuration. The implementation can choose ML-DSA, SLH-DSA, or other supported post-quantum signature systems. Key serialization and verification match the exact interface assumed by the formal model.

**Invariant protocol structure.** Changing the algorithm choices does not change the message formats or state transitions. This property aligns with the protocol description and ensures that the proofs apply for any approved parameter sets.

### 8.3 Constant Time Behavior

The implementation uses constant-time cryptographic operations for all operations that depend on secret data.

- KEM decapsulation uses constant-time implementations provided by the vendor library.
- Signature verification uses constant-time routines.
- RCS encryption and decryption avoid data dependent branches and table lookups.
- The comparison of authentication tags and transcripts uses constant-time memory comparison functions.

The implementation contains no branches or memory access patterns that depend on tunnel keys, KEM secrets, PSKs, or any intermediate KDF outputs. This conforms to the assumptions used in the security proofs.

### 8.4 RNG Requirements and Safety

DKTP requires a cryptographically secure pseudo-random number generator. The implementation uses the system CSPRNG and has the following properties:

- The RNG is invoked to generate ephemeral KEM key pairs.
- Nonces for handshake encryption are derived from KDF outputs and not from the RNG, ensuring determinism and uniqueness.
- Transport nonces are KDF-derived 32-byte initial values advanced by the RCS internal counter; sequence numbers are authenticated header fields, not nonce encodings.
- RNG failures are detected before the protocol continues.

The security proofs assume correct operation of the CSPRNG only for ephemeral KEM key generation.

### 8.5 Key and State Erasure

The implementation erases sensitive material at the appropriate times:

- ephemeral KEM private keys are cleared by the key-exchange reset or after their corresponding ratchet operation,
- directional shared secrets `secl` and `secr` are erased after tunnel key derivation,
- tunnel keys are overwritten when ratcheting produces new ones,
- updated PSKs replace old PSKs and overwrite their previous values,

- intermediate buffers used during RCS encryption and decryption are cleared.

The erasure functions used are resistant to compiler optimization removal and match the assumptions required to prevent key recovery by memory inspection.

## 8.6 Validation of Sequence Number and Nonce Handling

The implementation correctly enforces all sequence and nonce properties required by the formal model.

**Sequence numbers.** Each direction maintains an unsigned 64 bit sequence counter. The active receive path enforces strict increment-by-one acceptance: an inbound encrypted packet is accepted only when its sequence equals the current receive sequence plus one. The implementation defines a sequence terminator constant, but the modeled receive path does not rely on a separate replay window or independent wraparound theorem; deployments should impose a session lifetime or packet-count limit before wraparound becomes relevant.

**Nonces.** Nonces are KDF-derived initial values and are then advanced by RCS internal counter progression. Sequence numbers never repeat within an accepted receive stream and are authenticated through the packet header, but they are not used as RCS nonces.

**Associated data.** The entire header (flag, message length, sequence, timestamp) is passed as associated data to RCS. Any modification breaks the authentication tag, preventing replay and reordering attacks.

This completes the implementation conformance analysis and confirms that the DKTP implementation matches the formal model and specification in all cryptographically relevant details.

## 9 Concrete Security Estimates

This section provides concrete security estimates for DKTP based on the parameter sets commonly selected for the KEM and signature schemes, the strength added by mandatory directional pre-shared keys, and the 512-bit symmetric tunnel keys derived at the end of the handshake. Estimates rely on NIST post-quantum security categories and the theoretical bounds of symmetric and asymmetric cryptographic primitives.

### 9.1 Security Levels for Select KEM and Signature Choices

DKTP is designed to use a 512-bit symmetric-key profile for active secure channels. This profile uses 512-bit directional tunnel keys and cSHAKE-512 based key derivation. The authenticated encryption used in the tunnel is RCS-512. These choices provide a large symmetric margin, but the effective security of a complete session remains limited by the weakest applicable assumption in the configured KEM, signature scheme, KDF, authenticated-encryption construction, endpoint security, and key-management profile.

The asymmetric components of DKTP, namely the key encapsulation mechanism and the digital signature system, are selected through compile-time configuration. The protocol supports a variety of post-quantum families, including lattice-based, code-based, and hash-based systems. DKTP selects the asymmetric *family pairing* at compile-time (via `DKTP_CONFIG_DILITHIUM_KYBER`,

DKTP\_CONFIG\_DILITHIUM\_MCELIECE, or DKTP\_CONFIG\_SPHINCS\_MCELIECE), while the concrete parameter set and hence the NIST security category are governed by the underlying QSC library configuration (for example QSC\_DILITHIUM\_S5P87 and QSC\_KYBER\_S5K4P1024), not by DKTP itself. With the default QSC configuration, DKTP is built with ML-DSA-87 (Dilithium-S5) and ML-KEM-1024 (Kyber-S5), that is, NIST Category 5 asymmetric primitives providing approximately 256-bit post-quantum security, and emits the `dilithium-s5_kyber-s5_sha3_rcs` configuration string. Lower categories, and the Dilithium/McEliece and SPHINCS+/McEliece pairings, remain available by reconfiguring the QSC library without any change to DKTP’s message formats or to the proofs. The symmetric core remains 512-bit in every configuration, and the directional pre-shared keys are mixed in for all parameter selections.

The table below lists representative high-category parameter sets and their approximate security strengths. Lower parameter sets remain usable when their lower security target is acceptable to the deployment.

| Primitive        | Parameter Set       | Type         | Approximate Security Bits                 |
|------------------|---------------------|--------------|-------------------------------------------|
| ML-KEM           | ML-KEM-1024         | KEM          | $\approx 256$                             |
| Classic McEliece | mceliece8192128     | KEM          | $\approx 256$                             |
| ML-DSA           | ML-DSA-87 (Level 5) | Signature    | $\approx 256$                             |
| SLH-DSA          | SLH-DSA-SHAKE-256s  | Signature    | $\approx 256$                             |
| RCS              | RCS-512             | AEAD profile | implementation-defined symmetric margin   |
| cSHAKE           | cSHAKE-512          | KDF/XOF      | 256-bit collision / large preimage margin |
| SHA3             | SHA3-512            | Hash         | 256-bit collision / large preimage margin |

Table 1: Representative high-category primitive options for DKTP. Asymmetric entries are bounded by their selected NIST category or published security claim; symmetric entries describe design margins rather than a complete end-to-end channel-security level.

In summary, DKTP combines post-quantum asymmetric primitives (up to approximately 256-bit / NIST Category 5 strength, depending on the selected parameter set) with a large-margin symmetric transport profile. The tunnel keys, nonce derivation, PSK update mechanism, transcript hashing, and authenticated encryption rely on SHA3-512, cSHAKE-512, and RCS-512. The resulting channel-security claim is computational and assumption-dependent: it is bounded by the configured primitive profile, the PSK lifecycle, endpoint security, side-channel resistance, erasure, and operational key management.

## 9.2 Impact of PSK Strength on Overall Security

DKTP uses two persisted pre-shared secrets in the peering key structures:

`pssl` (local pre-shared secret),      `pssr` (remote pre-shared secret).

These values are mixed with the directional KEM shared secrets during channel key derivation:

$$(tckl, nl) \leftarrow \text{KDF}_{\text{DKTP}}(\text{secl}, \text{pssr}), \quad (tckr, nr) \leftarrow \text{KDF}_{\text{DKTP}}(\text{secl}, \text{pssl}).$$

**Security impact.** Stronger pre-shared secrets improve robustness against partial compromise scenarios by ensuring that channel key derivation retains secret entropy even if one source is



weakened. Conversely, weak or predictable pre-shared secrets reduce this benefit and shift effective security toward reliance on the secrecy of the KEM shared secrets alone.

**Update rule and persistence.** During key derivation, DKTP updates persisted peering secrets using cSHAKE-512:

$$\text{pssl} \leftarrow \text{cSHAKE512}(\text{pssl}, \perp, \text{tckl}[0:32]), \quad \text{pssr} \leftarrow \text{cSHAKE512}(\text{pssr}, \perp, \text{tckr}[0:32]).$$

The updated values are saved and seed subsequent handshakes.

### 9.3 Security Margins and Comparison to NIST Categories

NIST defines five categories of post-quantum security strength, with Category 5 corresponding to approximately 256-bits of security against classical attacks and 128-bits against quantum Grover style attacks.

In DKTP:

**Asymmetric layer.** The strength of the handshake depends on the selected KEM and signature scheme and typically ranges between Category 3 and Category 5.

**Symmetric layer.** The symmetric tunnel keys are 512-bits in each direction. Under quantum attack, Grover style search reduces symmetric security by a square root factor, leaving:

$$\text{Effective quantum symmetric security} \approx 256 \text{ bits.}$$

This exceeds NIST Category 5 requirements.

**RCS AEAD.** RCS uses a Rijndael based permutation and a cSHAKE based KDF and authentication layer. These have symmetric security levels aligned with the 512-bit tunnel keys and are far above Category 5.

**PSK contribution.** Because PSKs are included directly as KDF inputs, the symmetric strength is preserved across ratchet updates, even when asymmetric primitives are refreshed with lower category parameters.

**Overall channel security.** The final security level of a DKTP channel is:

- 512-bit symmetric strength when viewed against classical adversaries,
- approximately 256-bit symmetric strength against quantum adversaries,
- limited by the lower of the KEM and signature scheme during the initial handshake,
- increased after ratchet update because the PSK update is driven by 512-bit tunnel keys.

This provides a significant cushion above the highest NIST post-quantum category. DKTP retains strong security margins even when conservative parameter sets are chosen for asymmetric primitives.

## 10 Conclusion

The Dual Key Tunneling Protocol provides a high-assurance design for establishing authenticated and confidential channels under post-quantum primitive assumptions. This paper has presented an implementation-aligned formal analysis of the protocol profile. The handshake fields, key-derivation inputs, directional shared secrets, tunnel keys, ratchet updates, and transport packet format have been expressed both in engineering terms and through the formal security model.

The DKTP handshake uses a rolling 512-bit transcript hash to bind the complete exchange history into a consistent and authenticated structure. The transcript hash is initialized from the first message hash and updated at each subsequent handshake stage, ensuring that both parties agree on the same protocol parameters, peer identities, exchanged keying material, and message ordering. The transcript hash prevents any reordering or manipulation of the handshake and is explicitly verified at the establish phase through encrypted comparison.

The key schedule uses two independent KEM shared secrets together with directional pre-shared keys. These values are combined through domain separated cSHAKE-512 key derivation with per-channel identity labels to produce directional tunnel keys and initial nonces.

The transport phase uses RCS-512 authenticated encryption with KDF-derived initial nonces, authenticated headers, and strict replay and ordering rules. These features provide confidentiality, integrity, nonce uniqueness under RCS counter progression, and resistance to packet manipulation. Handshake PSK advancement updates the persisted PSK state after successful establishment, while the optional asymmetric ratchet refreshes active tunnel directions.

The security proofs show that DKTP achieves mutual authentication, handshake confidentiality, ciphertext integrity, replay resistance, forward secrecy, post-compromise security, and ratchet security. Each proof reduces one of these goals to the assumptions of the underlying KEM, signature scheme, KDF, and RCS authenticated encryption. The rolling transcript hash and mandatory directional PSKs play central roles in these reductions.

The implementation mapping confirms that the code follows the specification and the formal model. Cryptographic comparison operations use constant-time equality functions, sensitive temporary state is erased by the relevant reset or cleanup path, and the random number generator is used for ephemeral KEM key generation. The header structure, KDF-derived nonce initialization, sequence number checks, and packet layout match the formal description.

Taken together, these results demonstrate that DKTP is a robust, well structured, and conservatively designed secure channel protocol that provides strong protections against both classical and quantum adversaries. Its combination of rigid authenticity, strong symmetric encryption, algorithm agility, and ratchet based state evolution make it a suitable foundation for high security applications that require long-term confidentiality and resilience against compromise.

## References

1. National Institute of Standards and Technology. *FIPS 203: Module Lattice Based Key Encapsulation Mechanism Standard (ML KEM)*. U.S. Department of Commerce, 2024. Available at: <https://csrc.nist.gov/pubs/fips/203/final>.
2. National Institute of Standards and Technology. *FIPS 204: Module Lattice Based Digital Signature Standard (ML DSA)*. U.S. Department of Commerce, 2024. Available at: <https://csrc.nist.gov/pubs/fips/204/final>.
3. Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J. M., Schwabe, P., Seiler, G., and Stehlé, D. *CRYSTALS ML-KEM: Algorithm Specifications and Supporting Documentation*. 2021. Available at: <https://pq-crystals.org>.
4. Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., and Stehlé, D. *CRYSTALS ML-DSA: Algorithm Specifications and Supporting Documentation*. 2021.
5. National Institute of Standards and Technology. *FIPS 202: SHA 3 Standard, Permutation Based Hash and Extendable Output Functions*. U.S. Department of Commerce, 2015. Available at: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf>.
6. Kelsey, J., and National Institute of Standards and Technology. *SP 800 185: SHA 3 Derived Functions, cSHAKE, KMAC, TupleHash, and ParallelHash*. 2016. Available at: <https://doi.org/10.6028/NIST.SP.800-185>.
7. Bertoni, G., Daemen, J., Peeters, M., and Van Assche, G. *The Keccak Reference*. Submission to the NIST SHA 3 Competition, 2011. Available at: <https://keccak.team>.
8. Bellare, M., and Namprempre, C. *Authenticated Encryption: Relations among Notions and Analysis of the Generic Composition Paradigm*. Journal of Cryptology, 2008.
9. Rogaway, P., and Shrimpton, T. *A Provable Security Treatment of the Key Wrap Problem*. Eurocrypt, 2006.
10. Marlinspike, M., and Perrin, T. *The Double Ratchet Algorithm*. Signal Foundation, 2016.
11. Perrin, T. *The Noise Protocol Framework*. Revision 34, 2022. Available at: <https://noiseprotocol.org>.
12. Canetti, R., and Krawczyk, H. *Analysis of Key Exchange Protocols and Their Use for Building Secure Channels*. Eurocrypt, 2001.
13. LaMacchia, B., Lauter, K., and Mityagin, A. *Stronger Security of Authenticated Key Exchange*. ProvSec, 2007.
14. Dyer, J., Chen, S., and Shrimpton, T. *On the Security of TLS 1.3 Handshake Key Schedule*. IEEE S&P Workshops, 2018.
15. Underhill, J. G. *RCS Specification*. Quantum Resistant Cryptographic Solutions Corporation, 2025. Available at: <https://www.qrcscorp.ca>.
16. Underhill, J. G. *Dual Key Tunneling Protocol (DKTP) Technical Specification*. Quantum Resistant Cryptographic Solutions Corporation, 2025. Available at: <https://www.qrcscorp.ca>.
17. QRCS Corporation. *DKTP Reference Implementation*. Source repository, 2025. Available at: <https://github.com/QRCS-CORP/DKTP>.